

*What I was proud of was that I used very few parts to build a computer that could actually speak words on a screen and type words on a keyboard and run a programming language that could play games. And I did all this myself.*

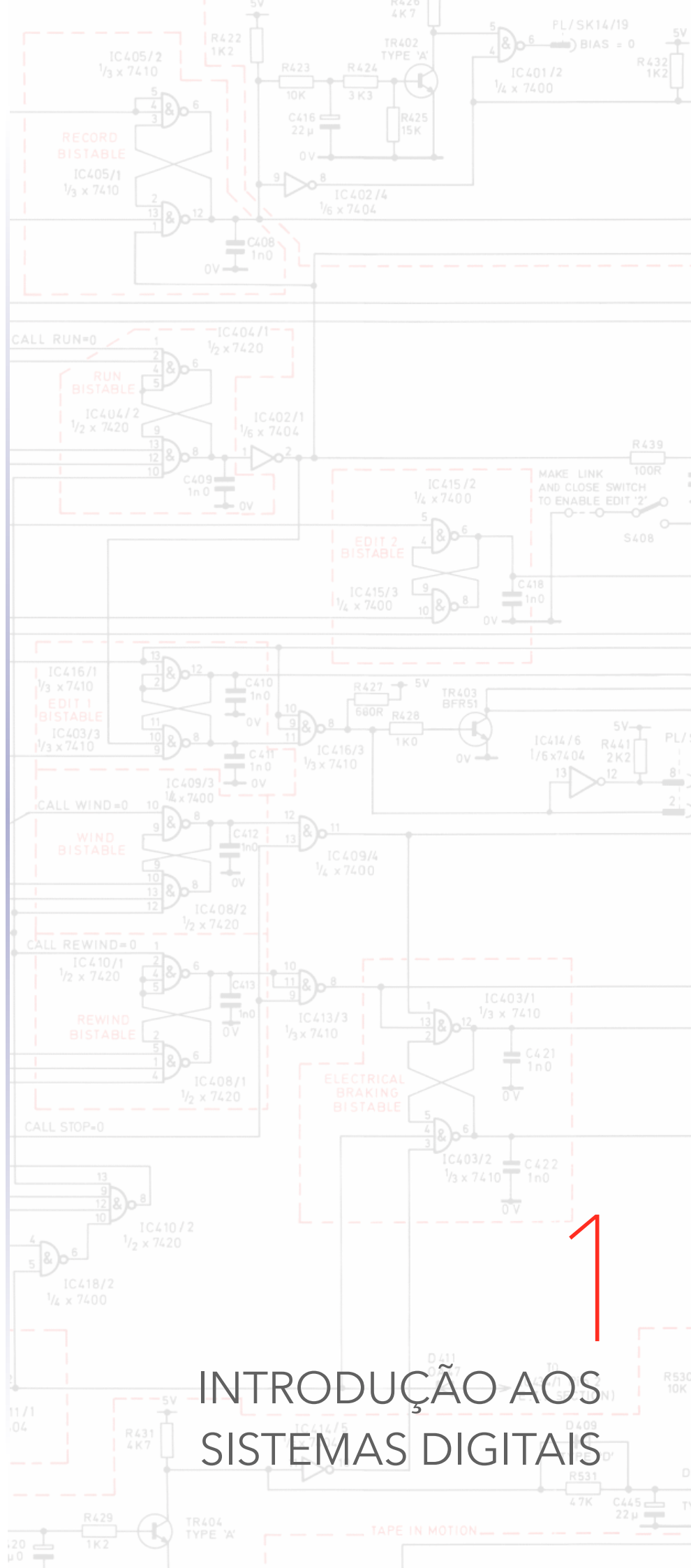
*Steve Wozniak*



universidade de aveiro  
theoria poiesis praxis

# INTRODUÇÃO AOS SISTEMAS DIGITAIS

1



# Atenção!

Todo o conteúdo deste documento pode conter alguns erros de sintaxe, científicos, entre outros... Não estudes apenas a partir desta fonte. Este documento apenas serve de apoio à leitura de outros livros, tendo nele contido todo o programa da disciplina de Introdução aos Sistemas Digitais, tal como foi lecionada, no ano letivo de 2013/2014, na Universidade de Aveiro. Este documento foi realizado por Rui Lopes.



Nos dias que correm, poucos são os momentos em que nós, seres humanos, não interagimos com ambientes digitais. Hoje o **mundo digital** é um espaço muito vasto e que compreende praticamente todas as ações humanas, tanto a nível social, como mental e até mesmo físico. Este chamado “mundo” tem levado uma grande transformação desde os últimos quarenta anos, produto do desenvolvimento de altas velocidades de circuitos integrados. Neste tempo conseguimos passar das **válvulas de vácuo** aos **chips**, este, que pode conter dezenas de milhões de **transístores** (componente eletrónico que serve como amplificador e interruptor de sinais elétricos) que podem ser programados de modo a criarem um sistema integrado num chip que, utilizando tecnologias do passado, conseguem conter vários circuitos *flip-flop* e portas num pequeno espaço físico.

**mundo digital**

**válvulas de vácuo, chips,  
transístores**

## 1. Ambiente Digital

Para se poder compreender como é que um computador ou outra máquina com potencialidades digitais funciona, temos que compreender como é que tais sistemas se sustentam: qual é a sua base. Para tal, precisamos de estudar desde o básico, de modo a que, no final, se possa construir, nem que através de um esquema, todo um sistema digital.

### Analógico versus Digital

Entre vários aparelhos que possamos ter e usar, eles podem ser **analógicos** ou **digitais**. Um **aparelho analógico** trabalha baseado num contínuo intervalo de valores de voltagem, ou outra quantidade. Um **aparelho digital** também, só com uma ligeira diferença: o intervalo de valores é descontínuo e varia entre apenas dois estados, aos quais damos o nome de **0** e **1** (significando o mesmo se fosse “baixo” e “alto”, “preto” e “branco”, etc...). A estes números damos o nome de **símbolos**, e não valores!

**aparelhos analógicos  
aparelhos digitais**

**0 e 1  
símbolos**

A era digital avançou, principalmente, através dos computadores, estes, que entraram no comércio público a partir da década de '60. Mas só nos últimos dez a vinte anos é que a era digital se focou também noutros aspetos da vida. Como exemplos, podemos tomar a câmara fotográfica, que outrora se utilizava usando uma fita onde, mecanicamente eram gravados os registos (informação), ou, em alguns locais, os semáforos, que costumavam funcionar através de um relógio e hoje em dia funcionam através de controladores e sensores na estrada que avaliam a densidade de tráfego. Mas porque é que hoje em dia há uma tão grande necessidade de utilizarmos aparelhos digitais? Por vários aspetos, entre eles:

- ▶ Eficácia e rendimento na produção de resultados - Dados os mesmos *inputs* que em analógico, um circuito digital efetua os mesmos cálculos, embora sem depender de muitos fatores físicos como a temperatura, voltagem, entre outros...;
- ▶ Facilidade de idealização - O processo-base da criação destes circuitos é a **lógica**, não necessitando de grandes habilidades matemáticas, sendo que o comportamento dos pequenos circuitos pode muito bem ser mentalmente visualizado sem a compreensão de componentes eletrónicos como transístores, condensadores, ...;
- ▶ Flexibilidade e funcionalidade - A partir do momento em que um problema foi implementado num raciocínio em ambiente digital, este, pode ser resolvido em pequenos passos lógicos.

# 3 INTRODUÇÃO AOS SISTEMAS DIGITAIS

- ▶ Programabilidade - Grande parte dos produtos digitais são criados, hoje em dia, gerando programas, também, em **linguagens de hardware descritivas (HDL)**. Estas linguagens permitem ambas estrutura e função de um circuito que sejam específicas ou *modeladas*. Para além de um compilador, um HDL típico também vem com sistemas de síntese e simulação. Estas ferramentas são usadas para testar o comportamento do hardware, antes deste ser realizado, e só depois é que é sujeito a uma síntese num circuito de testes particular.
- ▶ Velocidade - Os aparelhos digitais vencem a nível de velocidade. Transístores individuais e circuitos integrados podem funcionar em menos de 10ps (pico segundos), e examinar *inputs* e produzir *outputs* em menos de 1ns (nanossegundo). Isto significa que estes materiais são capazes de produzir cerca de um bilião de resultados ou mais, por segundo.
- ▶ Mais económico - Os circuitos digitais podem fornecer imensa funcionalidade num curto espaço físico. Estes circuitos costumam estar integrados em chips, o que proporciona mais espaço para mais semelhantes, aumentando o rendimento da máquina.

**HDL**

## Álgebra de Boole

De modo a compreender melhor como é que os bits são guardados e manipulados dentro de um computador, há que entender que o bit 0 corresponde a **falso** e o bit 1 corresponde a **verdadeiro**, o que nos permite trabalhar com **dados lógicos verdadeiro/falso**. Este tipo de operações pertencem à **álgebra de Boole**, em honra ao matemático George Boole (1815-1864), pioneiro da área da lógica matemática. Três operações básicas booleanas são as AND, OR e XOR (ou exclusivo). Estas operações são semelhantes às de MAIS e VEZES, dado que elas combinam pares numéricos (as parcelas - *input*) de forma a produzir um terceiro número (o resultado - *output*). Ao invés das comuns operações matemáticas, as operações de Boole trabalham com valores lógicos, em vez de valores numéricos.

A operação booleana AND está desenhada para deduzir a verdade ou a falsidade de uma conclusão, nela, combinada duas premissas pela conjunção *e* (*and*). Tais conclusões têm, assim, a forma

$$P \text{ AND } Q.$$

onde *P* representa uma premissa e *Q* representa outra premissa - por exemplo,

O Sapo Cocas é um sapo *e* a Miss Piggy é uma atriz.

Os *inputs* da operação AND representam a verdade ou a falsidade do resultado da mesma; o *output* representa a verdade ou a falsidade de si mesmo. Sendo que uma conclusão do género *P AND Q* é verdadeira apenas quando ambas premissas são verdadeiras, pode-se concluir que 1 AND 1 é igual a 1, enquanto que em todos os outros casos o resultado será 0.

De uma maneira semelhante, a operação OR é baseada nas premissas das conclusões com a forma

$$P \text{ OR } Q$$

**falso, verdadeiro**

**dados lógicos verdadeiro/  
/falso, álgebra de Boole**

onde, de novo,  $P$  representa uma premissa e  $Q$  outra premissa. Tais conclusões são verdadeiras quando **pelo menos uma** das premissas é verdadeira.

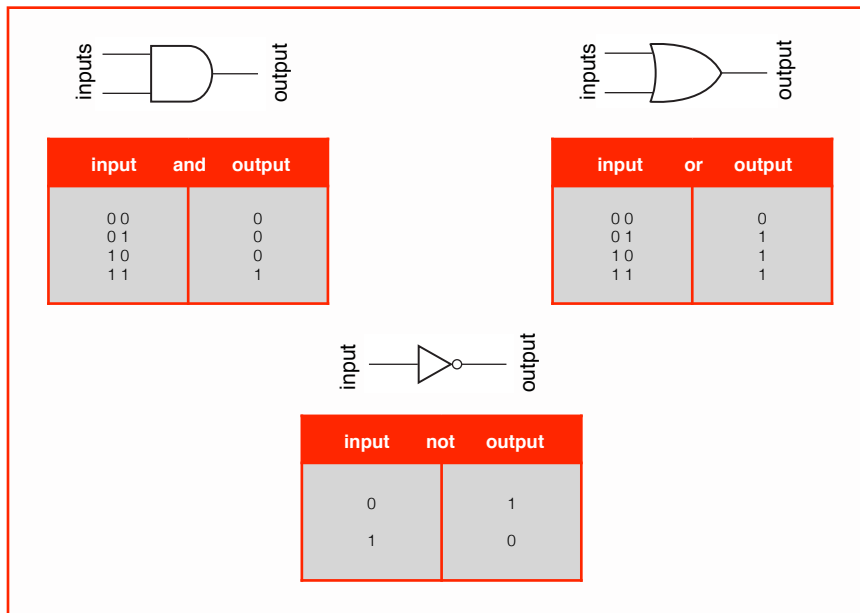
Não existe nenhuma conjunção no português que possa coincidir com o significado da operação XOR. A operação XOR produz um resultado de 1 (verdadeiro) quando um dos *inputs* é 1 (verdadeiro) e o outro é 0 (falso). Por exemplo, uma conclusão  $P \text{ XOR } Q$  significa “ $P$  ou  $Q$ , não ambos”. Em suma, a operação XOR dá um resultado de 1 quando os *inputs* são diferentes.

A operação NOT é uma outra operação booleana. Difere das operações AND, OR e XOR, porque apenas tem um e um só *input*. O seu resultado é sempre o contrário do seu *input*. Se o *input* de NOT é verdadeiro, então o *output* é falso, e vice-versa.

## Portas lógicas e flip-flops

O dispositivo que é capaz de produzir o resultado de uma dada operação booleana, quando se lhes são dados os *inputs*, é chamado de **porta** (*gate*). As portas podem ser contruídas de variados tipos de tecnologia tais como engrenagens, relés (interruptores eletromagnéticos), ou meros dispositivos óticos. Nos computadores dos dias de hoje, as portas estão implementadas em pequenos circuitos elétricos nos quais os valores 0 e 1 correspondem a diferentes valores de voltagem. Nós não nos precisamos de preocupar com esses valores. O que nos interessa, de momento, é representar estas portas, simbolicamente. Note-se que as diferentes portas são representadas por formas distintas, com os *inputs* representados num dos lados e o *output* representado no outro. As portas fornecem as bases à construção (o sustento) ao computador, para que ele funcione.

porta (*gate*)



**figura 1**  
representações simbólicas das portas AND, OR e NOT, seus *inputs* e *outputs*

Na página seguinte encontra-se um **circuito sequencial** (circuito eletrônico cujo comportamento é automaticamente afetado por entradas do passado) simbolicamente representado por operações booleanas. A isto se chama de **flip-flop**. Um flip-flop é, então, um circuito que produz um valor resultado de 0 ou 1, o qual permanece constante até ocorrer um pulso (mudança temporária de valor) de um outro circuito

circuito sequencial

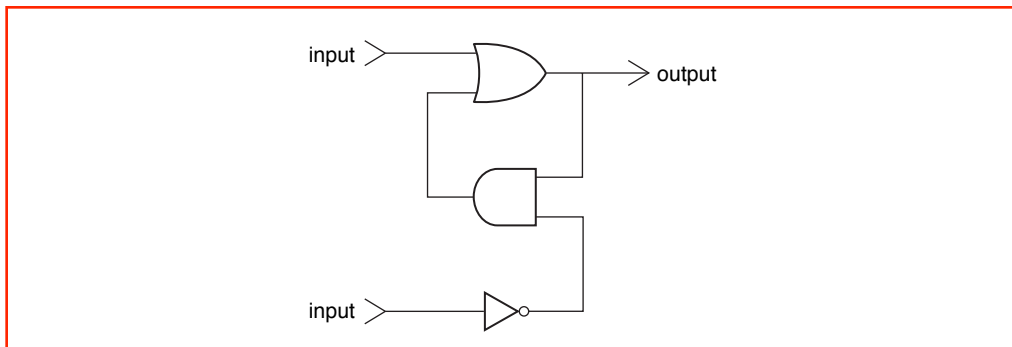
flip-flop

# 5 INTRODUÇÃO AOS SISTEMAS DIGITAIS

que provoca a alteração do *output*. Por outras palavras, o *output* alternar-se-á entre dois valores sobre o *feedback* de um circuito externo. Por outro lado, existe um outro tipo de circuitos cujo comportamento não é afetado por entradas passadas, mas sim por entradas atuais.

Neste circuito abaixo, enquanto ambos os *inputs* permaneçam 0, o *output* (quer seja 0 ou 1) nunca mudará. No entanto, se temporariamente colocarmos um 1 no *input* superior, o resultado será 1, enquanto que se colocarmos um 1 no *input* inferior, o resultado, forçosamente, será 0.

Vejamus com mais detalhe. Sem ter conhecimento do atual estado do *output*, imaginemos que o *input* superior é igual a 1, enquanto que o inferior é igual a 0. Isto fará com que o resultado da porta OR seja 1, seja qual for o valor atribuído para o



**figura 2**  
**um simples circuito flip-flop**  
**(sequencial)**

outro *input* desta porta. Assim, ambos os *inputs* da porta AND serão 1, dado que a outra porta perfaz o valor de 1 (a porta NOT traduz de 0 para 1). Por conseguinte, a porta AND resolve a operação e finaliza-a com 1, o que significa que o segundo *input* para a porta OR será 1. Isto garante que o *output* da porta OR permaneça 1, mesmo que o *input* superior seja alterado para 0. Em suma, o *output* do flip-flop torna-se 1, e o mesmo manter-se-á independentemente do valor do *input*.

O propósito do estudo dos flip-flops é importante dado três pontos. Primeiro, demonstra como os aparelhos dos dias de hoje podem ser construídos, todos, através destas pequenas portas e circuitos, um processo hoje designado por *design* de circuitos digitais, área de grande relevo da engenharia informática.

Segundo, o conceito de flip-flop introduz-nos o conceito do abstrato e como pode ser útil na computação e arquitetura de computadores.

Terceiro, porque, atualmente, o uso de flip-flops é um método para o armazenamento de bits, ou seja, o valor do *output* corresponde ao valor do bit, o qual o flip-flop guarda. Outros circuitos podem ajustar este valor enviando pulsos, e receber com *input* o *output* do primeiro. Assim, podem ser criados imensos circuitos no interior de um computador, e cada um guarda, assim, um bit de dados. A esta tecnologia dá-se o nome de **integração de larga escala** (*very large-scale integration (VLSI)*), que permite que vários circuitos eletrónicos sejam contruídos numa só placa, chamada **chip**. Os **chips** são placas com ferramentas abstratas dado que contêm milhões de flip-flops.

**VLSI**  
**chip**

## 2. Sistemas numéricos e códigos

Tal como estudado na unidade anterior, os circuitos digitais processam informação através de dados binários. Na nossa vida, pouca coisa se encontra em dados binários, ou até mesmo, num ponto de vista geral, em dados numéricos. Sendo assim, a pessoa que desenha tais circuitos tem de fazer uma correspondência direta entre o binário e os números que usamos na vida real. O interesse desta unidade é poder

aprender como é que várias unidades, mais especificamente, várias bases, se relacionam entre si e como é que os dados podem ser representados em informação.

## Elementos de um sistema de numeração

Um sistema de numeração é composto por vários elementos. Primeiro, de forma a se poderem representar num papel, ou noutra via de escrita, um sistema precisa de um **alfabeto**. Um alfabeto é, então, um conjunto de símbolos usados para representar valores, neste caso, numéricos. Tais valores numéricos têm uma lógica adjacente, essa chamada de **base**. A base de um sistema corresponde à variação de um algoritmo da relação entre os valores presentes. Aqui, iremos considerar, principalmente, as bases binária, octadecimal, decimal e hexadecimal (2, 8, 10 e 16, respetivamente), embora hajam muitas outras, entre as quais, as bases 24, 60 e complexa, que nós usamos frequentemente nos nossos relógios (base 24) e na trigonometria (base 60).

A representação de um sistema também necessita de algumas regras - **notação posicional**. Estas regras variam de sistema para sistema, sendo que basta que o **comprimento de representação**<sup>1</sup> - número de símbolos - seja alterado, para que estas mudem.

Se analisarmos, agora, a notação posicional podemos considerar que a uma determinada quantidade  $X$  podemos associar uma lista ordenada de símbolos ou dígitos  $d_k$ :

$$X = (d_{n-1}, d_{n-2}r^{n-2}, \dots, d_0, d_{-1}, \dots, d_{-p})_r$$

O valor de  $X$  resulta de a cada símbolo estar associado um peso dependente da respetiva posição:

$$X = \sum_{k=-p}^{n-1} d_k r^k = d_{n-1}r^{n-1} + d_{n-2}r^{n-2} + \dots + d_0 + d_{-1}r^{-1} + \dots + d_{-p}r^{-p}$$

com  $d_k$  sendo o alfabeto (intervalo  $\{0, \dots, r-1\}$ ,  $n+p$  os símbolos e  $r$  a base).

| número de símbolos | base             | alfabeto                        |
|--------------------|------------------|---------------------------------|
| 2                  | 2 (binária)      | 0,1                             |
| 8                  | 8 (octadecimal)  | 0,1,2,3,4,5,6,7                 |
| 10                 | 10 (decimal)     | 0,1,2,3,4,5,6,7,8,9             |
| 16                 | 16 (hexadecimal) | 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F |

Através deste último cálculo, podemos prever qual será o nosso alfabeto das bases que pretendemos estudar (base 2, 8, 10 e 16).

Para exemplificar, podemos fragmentar um número, de acordo com o cálculo anterior, e assim, convertê-lo para uma base decimal:

$$DACA1_{16} = 13 \times 16^4 + 10 \times 16^3 + 12 \times 16^2 + 10 \times 16^1 + 1 \times 16^0 = 896161$$

**alfabeto**

**base**

**notação posicional**

**comprimento de representação**

**equação 1**

**quantidade de símbolos  $d_k$**

**equação 2**

**valor de  $X$  por peso**

**tabela 1**

**bases e alfabetos conhecidos**

<sup>1</sup> Na prática temos um conjunto finito de símbolos de representação e devem existir restrições à gama de representação.

Se nos dirigirmos às bases 8 e 16, podemos comprovar que ambas têm uma **relação direta** com a base binária (2). Essa relação consiste no facto de um dígito octal ser coincidente com **três dígitos binários** (bits), enquanto que um dígito hexadecimal é coincidente com **quatro dígitos binários** (*nibble*).

**relação direta**  
três dígitos binários  
quatro dígitos binários

| base binária | base octadecimal | base decimal | base hexadecimal |
|--------------|------------------|--------------|------------------|
| 0            | 0                | 0            | 0                |
| 1            | 1                | 1            | 1                |
|              | 2                | 2            | 2                |
|              | 3                | 3            | 3                |
|              | 4                | 4            | 4                |
|              | 5                | 5            | 5                |
|              | 6                | 6            | 6                |
|              | 7                | 7            | 7                |
|              |                  | 8            | 8                |
|              |                  | 9            | 9                |
|              |                  |              | A                |
|              |                  |              | B                |
|              |                  |              | C                |
|              |                  |              | D                |
|              |                  |              | E                |
|              |                  |              | F                |

**tabela 2**  
bases binária, octadecimal,  
decimal e hexadecimal

Sabendo as relações diretas entre a base octal e hexadecimal com a binária, podemos, muitas vezes, fazer **substituição direta**.

**substituição direta**

$$\begin{aligned} \text{DACA1}_{16} &= 1101\ 1010\ 1100\ 1010\ 0001_2 \\ 2715_8 &= 010\ 111\ 001\ 101_2 \end{aligned}$$

Com a notação hexadecimal, podemos também reparar que se torna mais útil de utilizar, dado que, para palavras *multibyte*, em binário, o erro de leitura é muito grande e a extensão do número também, enquanto que em hexadecimal conseguimos inverter essa situação e escrever o mesmo número em pouco espaço, baixando assim, o erro de leitura. Tomemos como exemplo o primeiro número binário acima representado. Este número de vinte algarismos ocuparia muita memória no computador, ao contrário do número DACA1, que apenas ocupa uma pequena porção dela.

## Mudanças de base

Há uma grande necessidade de, por vezes, efetuarmos uma mudança de base, para que possamos fazer um câmbio entre o mundo real decimal e as instâncias de processamento binário, por exemplo.

Basicamente, nesta sub-unidade “Mudanças de base”, pretendemos saber como passar de uma representação em  $n$  dígitos e base  $r$  para uma representação em  $m$  dígitos em base  $s$ .

Em geral uma quantidade resulta da justaposição de uma componente inteira com uma componente fracionária. No caso de quantidades não inteiras deve adotar-se um critério que determine o número de dígitos significativos consistente com os **erros de representação** nas bases inicial e final. Os erros de representação são comuns, mas podem ser evitados. Para os evitar, e para generalizar, tomemos  $n$  como o número de dígitos fracionários na base  $r$  e  $m$  o número de dígitos fracionários na base  $s$ . Assim, interessa-nos determinar  $m$  tal que os erros de representação sejam o mais próximos possíveis.

**erros de representação**

$$|E_r| \leq \left| \frac{r}{2} r^{-(n+1)} \right| \approx |E_s| \leq \left| \frac{s}{2} s^{-(m+1)} \right|$$

$$s^{-m} \approx r^{-n} \therefore m \approx n \frac{\log r}{\log s}$$

**equação 3**

**comparação do erro de representação**

Se considerarmos que a mudança de base não deve trazer acréscimo de precisão, então devemos consultar o resultado da seguinte operação:

$$m = \left\lceil n \frac{\log r}{\log s} \right\rceil$$

**equação 4**

**determinação do erro de representação com operação chão (floor)**

Por exemplo:

$$12.345_8 = 1 \times 8 + 2 + \frac{3}{8} + \frac{4}{64} + \frac{5}{512}$$

$$m = \left\lceil 3 \frac{\log 8}{\log 10} \right\rceil = \lceil 2.7 \rceil = 3$$

$$12.345_8 = 10.45_{10}$$

O sistema aritmético humano assenta na base 10 (os primeiros seres começaram a contar até 10, pois tinham dez dedos com que contar). Por esta razão, para fazermos uma mudança da base  $r$  para a base  $s$  temos de processar um algoritmo baseado em divisões sucessivas (DIVS). Basicamente o que acontece é que para convertermos um número de base 10 para base 2, 8 ou 16, temos de dividi-lo sucessivamente por 2, 8 e 16, respetivamente, guardando os seus restos.

Por exemplo (na página seguinte):

2 O operador é denominado “operador chão” (floor) e trunca a quantidade para o número inteiro imediatamente inferior.

# 9 INTRODUÇÃO AOS SISTEMAS DIGITAIS

$$\begin{array}{r}
 432 \quad |2 \\
 \hline
 400 \quad 216 \quad |2 \\
 \hline
 32 \quad 200 \quad 108 \quad |2 \\
 \hline
 20 \quad 16 \quad 100 \quad 54 \quad |2 \\
 \hline
 12 \quad 16 \quad 8 \quad 40 \quad 27 \quad |2 \\
 \hline
 12 \quad 0 \quad 8 \quad 14 \quad 20 \quad 13 \quad |2 \\
 \hline
 0 \quad \quad 0 \quad 14 \quad 7 \quad 12 \quad 6 \quad |2 \\
 \hline
 \quad \quad \quad 0 \quad 6 \quad 1 \quad 6 \quad 3 \quad |2 \\
 \hline
 \quad \quad \quad \quad 1 \quad \quad 0 \quad 2 \quad 1 \\
 \hline
 \quad \quad \quad \quad \quad 1
 \end{array}$$

$$432_{10} = 110110000_2$$

Ou até mesmo com a base 8:

$$\begin{array}{r}
 1462 \quad |8 \\
 \hline
 800 \quad 182 \quad |8 \\
 \hline
 662 \quad 160 \quad 22 \quad |8 \\
 \hline
 640 \quad 22 \quad 16 \quad 2 \\
 \hline
 22 \quad 16 \quad 6 \\
 \hline
 16 \quad 6 \\
 \hline
 6
 \end{array}$$

$$1462_{10} = 2666_8$$

## Adição e Subtração binária

Tal como na base 10, na base binária também podemos efetuar certas operações, tais como a **adição** e a **subtração**. Para fazermos a adição e a subtração temos que seguir a tabela abaixo e ter atenção a somas do género 1+1.

**adição e subtração binária**

| $\pm$ | 0 | 1 |
|-------|---|---|
| 0     | 0 | 1 |
| 1     | 1 | 0 |

**tabela 3**

**adição e subtração binária**  
(1+1 com transporte)

$$\begin{array}{r}
 11101 \\
 + 11001 \\
 \hline
 110110
 \end{array}
 \quad \text{e} \quad
 \begin{array}{r}
 11001 \\
 - 101 \\
 \hline
 10100
 \end{array}$$

## Multiplicação binária

Na multiplicação também temos uma tabela à qual devemos obedecer. Tal como na multiplicação de base 10, na de base 2, sempre que se multiplica por zero o resultado é zero. Na mesma, usamos a técnica *shift and add* - algoritmo usualmente usado para efetuar as multiplicações em bases decimais (multiplicar algarismo a algarismo e depois somar).

| x | 0 | 1 |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |

**multiplicação binária**

**tabela 4**

**multiplicação binária**

$$\begin{array}{r}
 10111 \\
 \times 1101 \\
 \hline
 00000 \\
 10111 \\
 \hline
 010111 \\
 00000 - \\
 \hline
 0010111 \\
 10111 - - \\
 \hline
 01110011 \\
 10111 - - - \\
 \hline
 100101011
 \end{array}$$

## Representação de quantidades negativas

O sistema de notação posicional, por si, não é capaz de representar quantidades negativas. Em alternativa, usam-se outras convenções para as suas representações. Entre elas, o **sinal e módulo** e os **códigos de complemento**.

A convenção de sinal e módulo tem uma consistência muito própria com a notação posicional, embora tenha a necessidade de mais um bit, este, para representar o sinal. Infelizmente, este sistema é natural para os humanos, mas impraticável nos sistemas digitais, dado que existem ambiguidades próprias e o seu tamanho é  $n$  bits mais 1 (para o sinal).

Por sua vez, os códigos de complemento, sendo usados mais frequentemente, o complemento para um e para dois, traduzem-se num subconjunto da gama de representação absoluta que codifica as quantidades negativas. Este sistema já se torna possível e praticável em sistemas digitais, pois não necessita de um símbolo adicional para a representação do sinal. Outra vantagem deste sistema é que ele permite facilmente o processamento integrado da soma algébrica, ao contrário do sinal e módulo.

**sinal e módulo, códigos de complemento**

## Sinal e módulo

Como dito anteriormente, o sinal e módulo consiste numa representação numérica, em base 2, que permite a representação não só de números positivos como

# 11 INTRODUÇÃO AOS SISTEMAS DIGITAIS

de negativos. Este sistema, também como referido anteriormente, contém certas ambiguidades que fazem com que o processamento de dados com o próprio se torne caótico e cíclico: o zero tem duas representações.

Em suma, o sistema traduz-se da seguinte forma:

$$\begin{aligned}\text{número} &= \text{sinal módulo} \\ \text{sinal} &\rightarrow 0 \text{ quando é "+"}; 1 \text{ quando é "-"} \\ \text{módulo} &\rightarrow \text{representação posicional do número} \\ 6 &= 00110 \quad -6 = 10110 \\ 0 &= 00000 = 10000\end{aligned}$$

## Complemento para a base $r$

Com  $n$  dígitos e base  $r$ , define-se o **complemento para a base  $r$**  de uma quantidade  $X$ , como

$$RC(X) = r^n - X$$

onde  $RC$  significa complemento para a base (*radix complement*).

O que se pretende com este cálculo é substituir uma quantidade  $M - X$  pela soma  $M + RC(X) = M + r^n - X$ . Este procedimento produz resultados válidos desde que:

$$\begin{aligned}-\frac{r^n}{2} \leq M - X \leq \frac{r^n}{2} - 1 \\ \therefore \\ M - X = (M - X) \bmod r^n \quad (3)\end{aligned}$$

## Complemento para a base 2

Como referido anteriormente, os códigos de complemento são relevantes aquando de sistemas computacionais binários. Existem dois códigos mais usados: o complemento para 1 e para 2. O complemento para 2 de uma quantidade representada por  $n$  dígitos é:

$$-X = (2^n - X)$$

Por questões de simplificação, usamos o complemento para 1, representado por  $n$  dígitos, da seguinte forma:

$$-X = (2^n - X - 1) + 1$$

Importa reconhecer que  $(2^n - X - 1) = (X'_{n-1} \dots X'_0)$ , donde:

$$-X = (X'_{n-1} \dots X'_0) + 1$$

**complemento para a base  $r$**

**equação 5**

**definição de complemento para a base  $r$**

**equação 6**

**generalização da definição de complemento para a base  $r$**

**equação 7**

**definição de complemento para 2**

**equação 8**

**definição de complemento para 1**

## Overflow

Quando o resultado de uma soma de dois números positivos dá negativo ou a soma de dois negativos dá positivo, dá-se o nome de **overflow**. Geralmente isto ocorre porque a operação excedeu a gama de representação.

$$\begin{array}{r} -3 \quad 1101 \\ + -6 \quad +1010 \\ \hline -9 \quad 10111 = +7 \end{array}$$

Em suma, podemos criar a seguinte tabela:

| representação          |                |                                                                                                |                                     |
|------------------------|----------------|------------------------------------------------------------------------------------------------|-------------------------------------|
| 1 - Sinal e módulo     |                | 1 bit para sinal (0 para positivo, 1 para negativo) + x bits para número em notação posicional |                                     |
| 2 - Complemento para 1 |                | a) números positivos                                                                           | igual ao sinal e módulo             |
|                        |                | b) números negativos                                                                           | inverter o sinal e módulo           |
| 3 - Complemento para 2 |                | a) números positivos                                                                           | igual ao sinal e módulo             |
|                        |                | b) números negativos                                                                           | inverter o sinal e módulo e somar 1 |
| nº                     | sinal e módulo | complemento para 1                                                                             | complemento para 2                  |
| 7                      | 0111           | 0111                                                                                           | 0111                                |
| 6                      | 0110           | 0110                                                                                           | 0110                                |
| 5                      | 0101           | 0101                                                                                           | 0101                                |
| 4                      | 0100           | 0100                                                                                           | 0100                                |
| 3                      | 0011           | 0011                                                                                           | 0011                                |
| 2                      | 0010           | 0010                                                                                           | 0010                                |
| 1                      | 0001           | 0001                                                                                           | 0001                                |
| 0                      | 0000           | 0000                                                                                           | 0000                                |
| -0                     | 1000           | 1111                                                                                           |                                     |
| -1                     | 1001           | 1110                                                                                           | 1111                                |
| -2                     | 1010           | 1101                                                                                           | 1110                                |
| -3                     | 1011           | 1100                                                                                           | 1101                                |
| -4                     | 1100           | 1011                                                                                           | 1100                                |
| -5                     | 1101           | 1010                                                                                           | 1011                                |
| -6                     | 1110           | 1001                                                                                           | 1010                                |
| -7                     | 1111           | 1000                                                                                           | 1001                                |
| -8                     |                |                                                                                                | 1000                                |

tabela 5

representação de números binários

## Adição algébrica em complemento para um

Nesta operação podem ser realizadas através das regras de adição básicas binárias. Os transportes para além do bit mais significativo devem ser sinalizados e somados ao resultado, contando apenas os primeiros quatro dígitos.

## Adição algébrica em complemento para dois

Nesta operação também podem ser realizadas através das regras de adição básicas binárias. Os transportes para além do bit mais significativo não são sinalizados nem somados ao resultado, contando apenas os primeiros quatro dígitos.

## 3. Códigos

Um código é uma sequência de  $n$  bits, em que cada sequência representa um determinado valor. Esse determinado valor é também chamado de **palavra de código**. O comprimento  $n$  de um determinado código com  $m$  valores a codificar é calculado através da seguinte expressão:

$$n \geq \lceil \log_2(m) \rceil \quad (4)$$

palavra de código

equação 9

comprimento de uma palavra de código

## BCD (Binary Coded Decimal)

O **BCD** (*binary coded decimal*) serve para codificar algarismos decimais. É um código **regular** - o seu comprimento de palavra é fixo -, **ponderado** - cada um dos bits de palavras de código tem um determinado peso -, **descontínuo** - palavras contínuas diferem em mais de um bit -, **não cíclico** - a primeira e a última palavras não são adjacentes - e **redundante** - nem todas as possíveis combinações de bits são usadas.

BCD

código ponderado

código descontínuo

código não cíclico

código redundante

| nº | BCD (8421) |
|----|------------|
| 0  | 0 0 0 0    |
| 1  | 0 0 0 1    |
| 2  | 0 0 1 0    |
| 3  | 0 0 1 1    |
| 4  | 0 1 0 0    |
| 5  | 0 1 0 1    |
| 6  | 0 1 1 0    |
| 7  | 0 1 1 1    |
| 8  | 1 0 0 0    |
| 9  | 1 0 0 1    |

tabela 6

representação código BCD

$$25_{10} = 11001_2$$

$$25_{10} = 0010\ 0101_{BCD}$$

4 A operação teto (*ceil*) significa o valor inteiro imediatamente superior.

## Código AIKEN e Excess-3 (XS3)

O código AIKEN é um código BCD **autocomplementar**, ou seja, o complemento para nove de cada dígito decimal obtém-se, simplesmente, subtraindo por nove. Foi criado por Howard Hathaway Aiken, *digital designer* do computador *Harvard Mark I*, da IBM.

Por sua vez o **Excess-3** (XS3) é um código autocomplementar que se obtém pelo excesso de três.

**AIKEN, código autocomplementar**

**Excess-3 (XS3)**

| nº | AIKEN (2421) |
|----|--------------|
| 0  | 0 0 0 0      |
| 1  | 0 0 0 1      |
| 2  | 0 0 1 0      |
| 3  | 0 0 1 1      |
| 4  | 0 1 0 0      |
| 5  | 1 0 1 1      |
| 6  | 1 1 0 0      |
| 7  | 1 1 0 1      |
| 8  | 1 1 1 0      |
| 9  | 1 1 1 1      |

| nº | XS3     |
|----|---------|
| 0  | 0 0 1 1 |
| 1  | 0 1 0 0 |
| 2  | 0 1 0 1 |
| 3  | 0 1 1 0 |
| 4  | 0 1 1 1 |
| 5  | 1 0 0 0 |
| 6  | 1 0 0 1 |
| 7  | 1 0 1 0 |
| 8  | 1 0 1 1 |
| 9  | 1 1 0 0 |

**tabela 7**  
representação de código em  
AIKEN e XS3

## Código de Gray

O código de Gray é um código não ponderado, regular, contínuo, cíclico, não redundante e refletido, com uma **distância de Hamming** de 1. A distância de Hamming é a distância entre palavras consecutivas, logo, o número de bits que diferem duas palavras de código consecutivas.

O código de Gray de um bit é constituído apenas por duas palavras: 0 e 1. Num código de Gray de  $n$  bits, as primeiras  $2^{n-1}$  palavras são iguais às do código de Gray de  $n-1$  bits precedidas de um zero ("0") - na posição do bit mais significativo. As últimas  $2^{n-1}$  palavras são iguais às do código de Gray de  $n-1$  bits, escritas pela ordem inversa e precedidas de um um ("1") - na posição do bit mais significativo.

Se pretendermos obter uma palavra de um código de Gray de  $n$  bits a partir da palavra correspondente do código binário de  $n$  bits:

**distância de Hamming**

- ▶ Adicionamos à palavra do código binário um bit à esquerda e atribuímos-lhe o valor 0;
- ▶ Numeramos todos os bits do código binário da direita para a esquerda;
- ▶ Atribuímos o valor 1 ao bit  $i$  do código de Gray se os bits  $i$  e  $i+1$  da palavra binária são diferentes;
- ▶ Atribuímos o valor 0 ao bit  $i$  do código de Gray se os bits  $i$  e  $i+1$  da palavra binária são iguais.

Se pretendermos obter uma palavra de código binário de  $n$  bits a partir da palavra correspondente do código de Gray de  $n$  bits:

- ▶ Numeramos todos os bits do código de Gray da esquerda para a direita;
- ▶ Atribuímos o valor do bit um do código de Gray ao bit um do código binário;
- ▶ O bit  $i$  ( $i=2,3, \dots, n$ ) do código binário é igual à soma exclusiva (XOR) do bit  $i-1$  do código binário do bit  $i$  do código de Gray.

## Deteção de erros em sistemas digitais

Um erro em sistemas digitais corresponde à corrupção de dados de um valor correto, para um valor diferente. Estes erros podem ocorrer tanto em sistemas de transmissão de informação digital (sob forma de ruído), como em sistemas de armazenamento (como memória, discos rígidos).

Os erros podem ser categorizados em dois tipos: **erros singulares**, quando só um bit de dados é corrompido; os **erros múltiplos**, quando dois ou mais bits de dados são corrompidos. Este último tipo de erro é muito menos frequente que o primeiro.

Os erros são detetáveis. Um código permite a deteção de erros, caso a corrupção de uma palavra resulte numa nova palavra que não faz parte do mesmo código. Deve-se ter por convenção que um sistema que permite a deteção de erros só gera, transmite e guarda palavras de código válidas. Assim, se uma sequência de bits é uma palavra válida de código, é assumida como correta, caso contrário, se uma sequência de bits não for uma palavra de código, esta é assumida como não-válida. É possível detetar todos os erros singulares caso a distância mínima entre todas os possíveis pares de palavras de código seja menor ou igual a dois. A tal código dá-se o nome de **código de distância 2**.

Para detetar erros múltiplos são necessários códigos de distância maior que 2. De forma a criar um código de distância maior que 2, é necessário utilizar mais que um bit de paridade. Por exemplo, um código de distância 3 permite a deteção de 2 erros. A palavra inválida 1001011 (com um erro), está mais próxima da palavra 0001011 do que qualquer outra palavra de código. Torna-se assim possível corrigir o erro modificando a palavra recebida para a palavra mais próxima no código.

De uma forma geral, um código de distância  $m$  permite a correção de  $c$  erros.

$$c = \left\lfloor \frac{m-1}{2} \right\rfloor$$

O método de Hamming permite gerar códigos de distância 3. Aqui,  $b$  bits de informação são entrelaçados com  $p$  bits de paridade:

$$n = b + p$$

Os bits de paridade são colocados nas posições específicas do código resultante (aos quais correspondem a potências de 2). As restantes posições são ocupadas pelos bits de informação.

$$n = 2^p - 1 \quad b = 2^p - p - 1$$

**erros singulares**  
**erros múltiplos**

**código de distância 2**

**equação 10**  
**correção de erros por**  
**distância de Hamming**

**equação 11**  
**distância de código e bits de**  
**paridade**

**equação 12**  
**posição de bits de paridade e**  
**de bits de informação**

É possível adicionar mais um bit de paridade ao código de Hamming de distância 3 e criar, deste modo, um código de distância 4. Este último código permite a correção de um erro ou dois erros.

## Códigos CRC

Os **códigos CRC** (*Cyclic Redundancy Check*) são muito usados em discos rígidos e em redes de comunicação. Estes permitem detectar erros múltiplos que aparecem agrupados num bloco ou pacote de dados (tais erros são chamados **burst errors**). Um *burst* é um conjunto de  $b$  bits, que começa e acaba com um bit errado e que dista do próximo *burst*,  $b$  ou mais bits. Um código CRC de  $n$  bits aplicado a uma sequência de dados de comprimento arbitrário permite detetar qualquer sequência errada composta por até  $n$  bits.

O algoritmo para gerar um código CRC de  $n$  bits usa um gerador polinomial composto por  $n+1$  bits que funciona como divisor módulo 2 da mensagem original. O resto da divisão  $R$  é o código CRC gerado. Sejam  $M$  os dados a transmitir (com número arbitrário de bits),  $G$  o polinómio gerador de grau  $n$  (com  $n+1$  bits) e  $R$  o resto da divisão módulo 2 de  $M$  por  $G$  (com  $n$  bits). A mensagem a transmitir calcula-se através da soma  $M + R$ .  $G$  é conhecido por ambos receptor e transmissor. A escolha do  $G$  determina quais os tipos de erros que são detetados.

Por exemplo, imaginemos que  $M=11100110$ ,  $G=11001$  ( $n=4$ ) e CRC<sub>4</sub>:  $G=x^4+x^3+x^0$ . O algoritmo é o seguinte:

- Adicionar  $n$  zeros à parte menos significativa de  $M$ : 11100110 0000 (só no transmissor);
- Alinhar o polinómio gerador com o bit mais significativo de  $M$  que está a 1;
- Gerar uma nova palavra realizando a operação XOR entre  $M$  e  $G$ ;
- Se  $M$  resultante é diferente de 00...0, ir ao ponto 2;
- Se  $M = 00...0$ , os  $n$  bits menos significativos contêm  $R$ .

**códigos CRC**

**burst errors**

## 4. Álgebra de Boole

Tal como já foi referido atrás muito sumariamente, uma **álgebra de Boole** é uma estrutura matemática baseada num conjunto  $\{B, +, \cdot\}$ , que satisfaz o seguinte conjunto de postulados: fecho, comutatividade, elementos neutros, distributividade, complementaridade e cardinalidade.

Aqui tenhamos como conceito, de que um **sistema digital** é uma entidade que processa um conjunto finito de entradas  $x_i$ ,  $i=0, \dots, n-1$  para produzir um conjunto finito de saídas  $y_k$ ,  $k=0, \dots, m-1$ . Num contexto binário,  $x_i$ ,  $y_i$  pertencentes ao conjunto  $\{0,1\}$ .

**álgebra de Boole**

**sistema digital**

### Postulados

Vejamos os postulados, ponto por ponto, pela ordem apresentada anteriormente. Tal como estudámos, a álgebra de Boole satisfaz um conjunto de postulados, nos quais o **fecho** é um deles.

$$\forall b_1, b_2 \in B, \begin{cases} b_1 + b_2 \in B \\ b_1 b_2 \in B \end{cases}$$

**fecho**

**equação 13**

**fecho em B**

Este postulado garante que para ambas as operações deste espaço vetorial, estas são fechadas em B.

O segundo postulado é o da **comutatividade**.

$$\forall b_1, b_2 \in B, \begin{cases} b_1 + b_2 = b_2 + b_1 \\ b_1 b_2 = b_2 b_1 \end{cases}$$

**comutatividade**

**equação 14**

**comutatividade em B**

O terceiro postulado é o da **existência de elementos neutros**:

$$\begin{aligned} \exists b_0 \forall b \in B : b + b_0 &= b \\ \exists b_1 \forall b \in B : b b_1 &= b \end{aligned}$$

**existência de elementos neutros**

**equação 15**

**existência de elementos neutros em B**

O quarto postulado é o da **distributividade**:

$$\forall b_1, b_2, b_3 \in B, \begin{cases} b_1 + b_2 b_3 = (b_1 + b_2) \cdot (b_1 + b_3) \\ b_1 \cdot (b_2 + b_3) = b_1 b_2 + b_1 b_3 \end{cases}$$

**distributividade**

**equação 16**

**distributividade em B**

O quinto postulado é o da **complementaridade**:

$$\forall b \exists b' \in B, \begin{cases} b + b' = b_1 \\ b b' = b_0 \end{cases}$$

**complementaridade**

**equação 17**

**complementaridade em B**

O sexto e último postulado é o da **cardinalidade**:

$$\# B \geq 2 \rightarrow \forall b \in B \exists a \in B : a \neq b$$

**cardinalidade**

**equação 18**

**cardinalidade em B**

## Valores, Operadores e Expressões

Num contexto binário temos naturalmente o conjunto de valores definido por  $B = \{0, 1\}$ . Circuitos eletrônicos elementares funcionando em modo comutado (*switched*), implementam os operadores lógicos elementares de acordo com as regras usuais da lógica matemática. As **expressões** são um conjunto de variáveis e/ou constantes associadas por operadores, por exemplo, a que se segue:

$$x + y \cdot (u \cdot v' + z)$$

**expressões**

**equação 19**

**exemplo de expressão booleana**

## Dualidade

O **princípio da Dualidade** fundamenta-se da seguinte forma: toda a expressão formada pelas variáveis  $a, b, c, \dots$  mais os elementos 0 e 1 e que envolva as operações de soma lógica, de produto lógico e de complementação possui uma expressão **dual** que se obtém trocando cada soma por um produto lógico e cada produto por uma soma lógica, e ainda os 0's por 1's e vice-versa.

**princípio da Dualidade**

**dual**

$$(a + a \cdot b)^D = a \cdot (a + b) \quad (a \cdot 1' + a' \cdot 1)^D = (a + 0') \cdot (a + 0')$$

**equação 20**

**princípio da Dualidade**

## Teoremas

Os teoremas são asserções acerca da validade da equivalência de expressões booleanas. Estas são demonstráveis a partir de postulados e/ou outros teoremas. Também o podem ser através de indução completa. O princípio da dualidade garante que uma vez demonstrada a validade de um teorema fica demonstrada a validade do respetivo dual.

O primeiro teorema a ser estudado é o da **unicidade do elemento neutro**. No conjunto B há apenas um e um só elemento  $b_0$  tal que  $b+b_0=b$  e apenas um e um só elemento  $b_1$  tal que  $b.b_1=b$ .

A **idempotência**, é um outro teorema que nos garante que qualquer que seja o  $b$  pertencente a B,  $b+b=b$  e  $b.b=b$ .

Um terceiro teorema, diz-nos que qualquer que seja o  $b$  pertencente ao nosso conjunto B,  $b+b_1=b_1$  e  $b.b_0=b_0$ . Este é o teorema do **elemento absorvente**.

O teorema da **absorção**, garante-nos que qualquer que seja o nosso  $x$  e  $y$  pertencentes ao B,  $x+xy=x$ . Semelhante a este, o teorema da **simplificação** diz que  $x+x'y=x+y$ .

Os dois teoremas que se seguem são os da **adjacência** que diz que, com as mesmas considerações que o anterior,  $xy+xy'=x$ , e o da **involução**, no qual qualquer que seja  $x$  pertencente a B,  $(x')'=x$ .

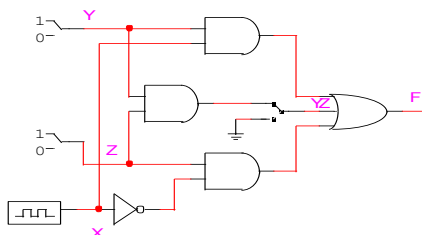
A **propriedade associativa** também é um teorema para que qualquer que seja  $x$ ,  $y$ ,  $z$  pertencentes a B,  $x(yz)=(xy)z$ .

Já as **Leis de De Morgan** são um teorema mais extenso, no qual,  $(x+y)'=x'.y'$ ,  $(xy)'=(x'+y')$ .

Finalmente, o nosso último teorema em estudo é o do **consenso**. Este teorema necessita de uma maior atenção na sua explicação. O teorema do consenso diz-nos que, quaisquer que sejam os  $x$ ,  $y$ ,  $z$  pertencentes ao nosso conjunto B,  $x.y+x'.z+y.z=x.y+x'.z+(x+y).z=x.y+x'.z$ . Demonstramos então, a sua veracidade:

$$\begin{aligned} x.y+x'.z+y.z &= x.y+x'.z+y.z(x+x') = \\ &= x.y+x'.z+y.z.x+y.z'.x = \\ &= x.y(b_1+z)+x'.z(b_1+y) = \\ &= x.y+x'.z \end{aligned}$$

Na prática, o teorema do consenso pode ser representado através do seguinte circuito:



**unicidade do elemento neutro**

**idempotência**

**elemento absorvente**

**absorção**

**simplificação**

**adjacência**

**involução**

**propriedade associativa**

**leis de De Morgan**

**consenso**

**equação 21**

**demonstração do teorema do consenso**

**figura 3**

**consenso na prática**

## Operadores compostos

Existem outros operadores, compostos, como são exemplo os dois que vamos estudar a seguir: o **operador XOR** e o **operador implicação**.

**operador XOR, operador implicação**

O operador XOR (em inglês *exclusive-or*), o qual significa “ou” exclusivo, o que faz com que o seu resultado possa ser ou  $x$  ou  $y$ , mas nunca ambos. Assim, entre  $x$  e  $y$ , o XOR é um operador que provoca que seja verdade quando temos  $x'y$  ou  $xy'$ . O operador XOR tem o seguinte símbolo  $\oplus$  e provoca então a seguinte igualdade:

$$x \oplus y = x.y' + x'.y$$

**equação 22**

**definição da operação XOR**

Por sua vez, o operador implicação tem o símbolo seguinte  $\rightarrow$  e tem como possível leitura “se... então”. A tabela de verdade é a seguinte:

| x   | $\rightarrow$ | y |
|-----|---------------|---|
| 0 0 |               | 1 |
| 0 1 |               | 0 |
| 1 0 |               | 1 |
| 1 1 |               | 1 |

**tabela 8**

**tabela de verdade da operação implicação**

Outros conjuntos completos de operadores, a partir dos quais se pode representar toda e qualquer relação booleana, são, por exemplo, os seguintes:

- $\{+, ., ' \}$ ;
- $\{+, ' \}$ ;
- $\{., ' \}$ ;
- NAND:  $a.b + c.d = (a.b + c.d)'' = ((a.b)'.(c.d)')'$ ;
- NOR:  $(a+b).(c+d) = [(a+b).(c+d)]'' = ((a+b)'+(c+d)')'$ .

## Funções booleanas

Uma **função booleana** é uma regra (correspondência) que associa um elemento do conjunto  $B=\{0,1\}$  a cada uma das  $2^n$  combinações possíveis que  $n$  variáveis independentes podem assumir. Tanto o domínio como o contradomínio são conjuntos enumeráveis e finitos de vetores binários.

**função booleana**

Vejamos uma generalização das leis de De Morgan:

$$f(x_0, \dots, x_{n-1}, 0, 1, +, .)' = f(x'_0, \dots, x'_{n-1}, 1, 0, ., +)$$

**equação 23**

**generalização das leis de De Morgan**

Assim, como exemplos, tenhamos os seguintes:

$$\begin{aligned} f(x, y, z) &= x.y(z' + y + x') + z.x(y + z') \\ f(x, y, z)' &= [(x' + y' + z.y'x)].(z' + x + y'.z) \end{aligned}$$

**equação 24**

**exemplo de aplicação**

Tomemos também uma aplicação do princípio da dualidade, mas a funções:

$$f(x_0, \dots, x_{n-1}, 0, 1, +, .)^D = f(x'_0, \dots, x'_{n-1}, 1, 0, ., +)$$

**equação 25**

**aplicação do princípio da dualidade**

Assim, como exemplos, tenhamos os seguintes:

$$\begin{aligned} f(x, y, z) &= x.y(z' + y + x') + z.x(y + z') \\ f(x, y, z)^D &= [(x + y + z'.y.x')].(z + x' + y.z') \end{aligned}$$

**equação 26**

**exemplo de aplicação**

Como podemos reparar, existe uma forte relação entre complementaridade e dualidade:

$$f(x_0, \dots, x_{n-1}, 0, 1, +, \cdot)' = f(x_0, \dots, x_{n-1}, 0, 1, +, \cdot)^D$$

Essa relação de igualdade pode ser usada como no seguinte exemplo:

$$\begin{aligned} f(x, y, z) &= x \cdot y' + x' \cdot y + y' \cdot z \\ f(x', y', z') &= x' \cdot y + x \cdot y' + y \cdot z' \\ f(x', y', z')^D &= (x' + y) \cdot (x + y') \cdot (y + z') \\ &= f(x, y, z)' \end{aligned}$$

**equação 27**  
relação entre  
complementaridade e  
dualidade

**equação 28**  
demonstração da relação  
anterior

A **representação** de funções booleanas é feita através de tabelas e é única. Uma dada função  $f$  tem uma única **tabela de verdade**. Do mesmo modo, uma dada função booleana admite múltiplas **representações algébricas**. Uma representação algébrica inclui frequentemente, termos redundantes. A eficiência na implementação técnica destas funções obrigará, automaticamente, ao uso de processos de simplificação.

**representação**  
**tabela de verdade**  
**representações algébricas**

## Formas canónicas

Para passarmos de uma representação tabular para uma representação algébrica podemos e devemos dar uso a um novo conceito - o de **formas canónicas**. Para definirmos uma forma canónica, podemos, para já, dizer que se trata de uma forma, não mínima, de representar funções booleanas. Uma forma canónica, entre quatro tipos possíveis, é uma soma ou um produto de **termos mínimos** ou **termos máximos**. Um termo mínimo, ou produto canónico de ordem  $k$ ,  $m_k$  é o produto lógico das  $n$  variáveis booleanas independentes, em que cada uma delas aparece uma e uma só vez, com o seu próprio valor ou complementada consoante toma valores 1 ou 0, respetivamente, na  $k$ -ésima combinação das variáveis independentes. Por outro lado, um termo máximo, ou soma canónica de ordem  $k$ ,  $M_k$  é a soma lógica das  $n$  variáveis booleanas independentes, em que cada uma delas aparece uma e uma só vez, com o seu próprio valor ou complementada consoante toma valores 0 ou 1, respetivamente, na  $k$ -ésima combinação das variáveis independentes.

**formas canónicas**

**termos mínimos**  
**termos máximos**

Recorrendo à definição dos termos canónicos, temos que a **1ª forma canónica** (soma de produtos - SOP), ou forma disjuntiva normal é dada pela seguinte forma:

**1ª forma canónica**

$$f(x_{n-1}, x_{n-2}, \dots, x_0) = \sum_{k=0}^{2^n-1} m_k \cdot f_k$$

**equação 29**  
**primeira forma canónica**

Na prática, ao identificar os 1's na saída de uma tabela de verdade (termos mínimos), ou seja, ver quais são os  $f_k=1$  e somar os respetivos termos mínimos  $m_k$ , estamos a avaliar a primeira forma canónica:

$$f(x_{n-1}, x_{n-2}, \dots, x_0) = \sum_{k=0}^{2^n-1} m_k$$

**equação 30**  
**primeira forma canónica**  
**revisitada**

Mais uma vez, recorrendo às definições de termos canónicos temos que a **2ª forma canónica** (produto de somas - POS), ou forma conjuntiva normal, é dada pela seguinte igualdade:

$$f(x_{n-1}, x_{n-2}, \dots, x_0) = \prod_{k=0}^{2^n-1} (M_k + f_k)$$

**2ª forma canónica**

**equação 31**

segunda forma canónica

Na prática, ao identificar os 0's na saída de uma tabela de verdade (termos máximos), ou seja, ver quais são os  $f_k=0$  e somar os respetivos termos máximos  $M_k$ , estamos a avaliar a segunda forma canónica:

$$f(x_{n-1}, x_{n-2}, \dots, x_0) = \prod_{k=0}^{2^n-1} M_k$$

**equação 32**

segunda forma canónica revisitada

Partindo da 1ª forma canónica e após dupla negação, obtemos a **3ª forma canónica**, uma implementação com NAND's:

**3ª forma canónica**

$$\begin{aligned} f(x_{n-1}, x_{n-2}, \dots, x_0) &= f(\dots)'' \\ &= \left( \sum_{k=0}^{2^n-1} m_k \cdot f_k \right)'' \\ &= \left( \prod_{k=0}^{2^n-1} m'_k \right)' \end{aligned}$$

**equação 33**

terceira forma canónica como dupla negação da primeira

Partindo da segunda forma canónica e após dupla negação, temos que:

$$\begin{aligned} f(x_{n-1}, x_{n-2}, \dots, x_0) &= f(\dots)'' \\ &= \left( \prod_{k=0}^{2^n-1} (M_k + f_k) \right)' \\ &= \left( \sum_{k=0}^{2^n-1} M'_k \right)' \end{aligned}$$

**equação 34**

quarta forma canónica como dupla negação da segunda

Admitindo uma especificação completa da tabela de verdade, a lista dos produtos canónicos implica a lista das somas canónicas. Por exemplo,

$$f(x_2, x_1, x_0)' = \sum m(3, 4, 5, 7) = \prod M(0, 1, 2, 6)$$

**equação 35**

implicação entre produtos e somas canónicas

Por outro lado se, por exemplo, a função tivesse os termos trocados, ficaria:

$$f(x_2, x_1, x_0)' = \sum m(0, 1, 2, 6) = \prod M(3, 4, 5, 7)$$

**equação 36**

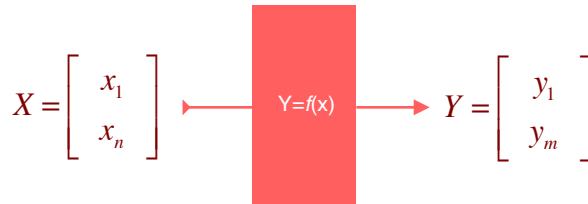
implicação entre produtos e somas canónicas (simetria)

Pode acontecer que nem sempre todas as combinações de entradas são passíveis de ocorrer. Pode acontecer, também, que não interesse especificar a saída para várias combinações de entrada. As entradas para as quais não se especifica, à

partida a saída, dizem-se irrelevantes. As combinações irrelevantes podem contribuir para a minimização da(s) forma(s) booleana(s). As funções booleanas assim definidas dizem-se incompletamente especificadas.

## 5. Minimização de funções booleanas

Como já tivemos oportunidade de estudar, uma função booleana é uma regra (correspondência) que associa um elemento do conjunto  $B=\{0,1\}$  a cada uma das  $2^n$  combinações possíveis que as variáveis independentes podem assumir.



**figura 4**  
um sistema combinatório

Também estudámos uma passagem sistemática da descrição por tabela de verdade para uma descrição algébrica. Existem implementações a dois níveis: a primeira forma canónica - soma de produtos (SOP) -, a segunda forma canónica - produto de somas (POS) -, terceira forma canónica - dupla negação da primeira forma canónica (NAND-NAND) - e quarta forma canónica - dupla negação da segunda forma canónica (NOR-NOR). Na maioria das vezes as formas canónicas não são as realizações mais simples em número de portas lógicas e no número de entradas de cada porta (**fan-in**).

Uma **simplificação** é recomendada, dado que com ela, muitas vantagens estão ligadas. Entre elas, a redução da complexidade, ao nível da implementação - redução do número de literais ou entradas lógicas (menos entradas implica portas mais rápidas em algumas tecnologias e os *fan-ins* são tipicamente limitados), redução do número de portas (número de portas (ou componentes) influencia custos de produção e os atrasos mínimos requerem normalmente mais portas) e redução do número de níveis lógicos (menor número de níveis lógicos implica menos tempo de propagação) -, ao nível de métodos tradicionais - reduzir o atraso à custa do aumento do número de portas - e de métodos modernos - equilíbrio entre a relação atraso *versus* número de portas.

O objetivo principal, então, da simplificação algébrica de uma função booleana  $f$  de  $n$  variáveis é encontrar uma função  $g$ , equivalente a  $f$  e que minimize um determinado critério de custo. Assim, é conveniente que haja: um número mínimo de instâncias de variáveis; um número mínimo de instâncias de variáveis numa soma de produtos (ou produto de somas); um número mínimo de termos numa soma de produtos (produto de somas) desde que não exista uma outra expressão com o mesmo número de termos e com menos instâncias de variáveis.

A simplificação algébrica é então um recurso pertinente mas muitas vezes não sistemático aos teoremas da álgebra de Boole. Simplifiquemos então uma função exemplo, como a seguinte:

$$\begin{aligned} f(x,y,z,w) &= x\bar{y} + z + \bar{x}\bar{z}w + y\bar{z}w = \\ &= x\bar{y} + z + \bar{z}(\bar{x}w + yw) = \\ &= x\bar{y} + z + w(\bar{x} + y) = \\ &= x\bar{y} + z + w(\overline{x\bar{y}}) = \\ &= x\bar{y} + z + w \quad (\text{forma irredutível}) \end{aligned}$$

**equação 37**  
simplificação a forma  
irredutível

## Expressões irreduzíveis

A abordagem “manual” pode conduzir a função mais simples e **irreduzíveis**, mas não necessariamente mínimas.

Todas as expressões são irredundantes (irreduzíveis). A supressão de qualquer termo ou instância de variável conduz a uma função diferente.

$$\begin{aligned} f &= \bar{x}y\bar{z} + \bar{x}\bar{y}z + x\bar{y}\bar{z} + \bar{x}yz + x\bar{y}z + xyz = \\ &= \bar{x}y\bar{z} + \bar{y}z(x + \bar{x}) + yz(x + \bar{x}) + x\bar{y}z = \\ &= \bar{z}(\bar{x}y + \bar{y}) + z(y + x\bar{y}) = \\ &= \bar{z}(\bar{x} + \bar{y}) + z(y + x) = \\ &= \bar{z}\bar{x} + \bar{z}\bar{y} + zy + zx \end{aligned}$$

equação 38

exemplo de solução (1 de 3)

Mais uma vez, expressões irreduzíveis podem não ser mínimas. Pode existir mais que uma expressão mínima. Vejamos o exemplo seguinte, com a mesma função que a anterior:

$$\begin{aligned} f &= \bar{x}y\bar{z} + \bar{x}\bar{y}z + x\bar{y}\bar{z} + \bar{x}yz + x\bar{y}z + xyz = \\ &= \bar{x}\bar{z}(y + \bar{y}) + x\bar{y}(z + \bar{z}) + yz(x + \bar{x}) = \\ &= \bar{x}\bar{z} + x\bar{y} + yz \end{aligned}$$

equação 39

exemplo de solução (2 de 3)

É, assim, conveniente desenvolver processos que produzam o conjunto de todas as expressões mínimas para seleção segundo outros critérios. Vejamos um terceiro exemplo, no qual se experimenta uma outra solução do nosso exercício:

$$\begin{aligned} f &= \bar{x}y\bar{z} + \bar{x}\bar{y}z + x\bar{y}\bar{z} + \bar{x}yz + x\bar{y}z + xyz = \\ &= \bar{x}y(z + \bar{z}) + xz(y + \bar{y}) + \bar{y}z(x + \bar{x}) = \\ &= \bar{x}y + xz + \bar{y}z \end{aligned}$$

equação 40

exemplo de solução (3 de 3)

## Adjacência e simplificação

Como processo de seleção para simplificação de funções booleanas, vejamos um fundamento, o **teorema da adjacência**, muito usado nos três exemplos dados anteriormente. Aqui, para todo o  $x$  e  $y$  pertencentes ao nosso conjunto B,  $xy + xy' = x$ . Para presenciar este passo, demonstremo-lo com dois simples exemplos:

$$\begin{aligned} f &= ab' + ab = a(b' + b) = a \\ g &= a'b' + ab' = b'(a' + a) = b' \end{aligned}$$

equação 41

aplicação do teorema da adjacência

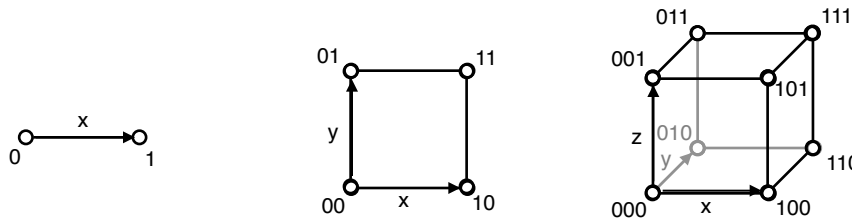
## Cubos Booleanos

Uma técnica para identificar quando é que se pode aplicar o teorema da adjacência é usando **cubos booleanos**. Esta é também uma forma alternativa à tabela de verdade. Assim, um cubo de  $n$  variáveis cada vértice, é adjacente a  $n$  vértices, isto é, para cada termo mínimo existem outros  $n$  termos mínimos que partilham  $n-1$  variáveis e diferindo, portanto, apenas numa variável.

De forma a fazermos o mapeamento das tabelas de verdade em cubos booleanos, identificamos três tipos de geometria de vértices: ON-set - vértices a cheio; OFF-set - vértices vazios; DC-set - vértices tipo X.

cubos booleanos

Para já vejamos três tipos de cubos, por número  $n$  de variáveis por vértice:



**figura 5**  
cubos booleanos: cubo-1,  
cubo-2 e cubo-3

Agora vejamos exemplos de geometria de vértices dos cubos:

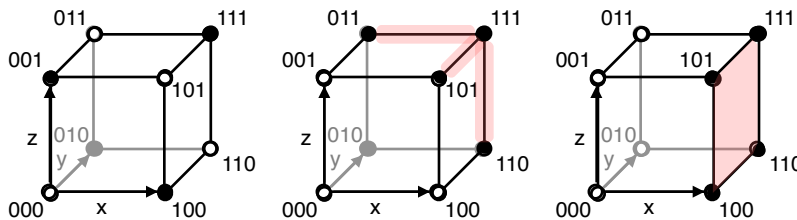


**figura 6**  
análise de cubo-2

No primeiro cubo-2 existem duas variáveis, designadas por A e por B. A variável A está constante e igual a “1”, enquanto que no segundo cubo-2 a variável B é constante e sempre “0”. Já os valores de B e de A, para os primeiro e segundo cubo-2, respetivamente, circulam perante um *loop*. A estes cubos dá-se-lhes o nome de **cubos de dimensão  $n-1$**  ou **subcubos**.

**cubos de dimensão  $n-1$**

Já os cubos-3 são diferentes. Vejamos os seguintes exemplos:



**figura 7**  
análise de cubo-2

Analisemos exemplo a exemplo. No primeiro caso é impossível explorar a adjacência (teorema), dado que as formas canónicas são irreduzíveis. Fazendo uma tabela de verdade, podemos, mais facilmente, comprovar que a função do primeiro cubo-3 é igual a  $x \oplus y \oplus z$ . No segundo caso existem três casos de adjacência, os quais se podem comprovar através da contiguidade dos pontos em 101, 111, 011 e 110. Já o terceiro caso tem uma adjacência sucessiva em quatro vértices (geometricamente formando um plano). Aqui apenas se mantém o  $x$ ,  $y$  e  $z$  são eliminadas.

Então já podemos dizer que um **subcubo  $n-m$**  é um agrupamento de  $2^m$  vértices, cada um deles adjacente a  $m$  vértices na coleção. Por exemplo, um subcubo 3-0, um vértice gera um mintermo (produto), com três literais; um subcubo 3-1, uma aresta, gerando um produto com dois literais; um subcubo 3-2, uma face, gerando um produto com um literal; um subcubo 3-3, um cubo, gerando a constante “1”. Em geral, um subcubo- $m$  dentro de um cubo- $n$  ( $m < n$ ) gera um produto com  $n-m$  literais.

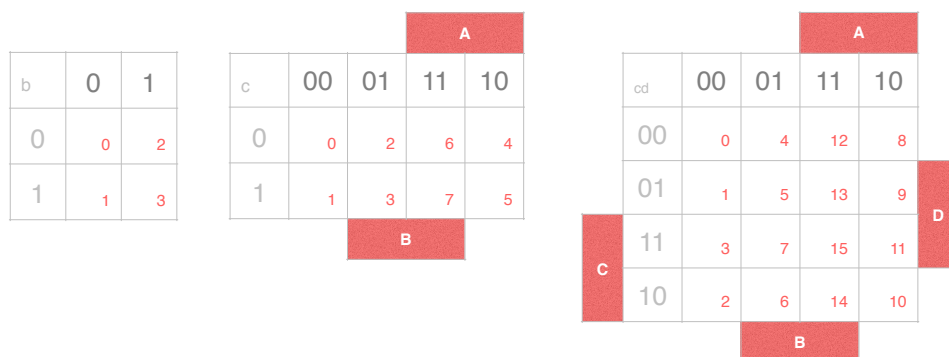
**subcubo  $n-m$**

## Métodos dos mapas de Karnaugh (K-map)

Para  $n > 3$  é praticamente impossível a utilização gráfica dos cubos booleanos para identificar quando se pode aplicar o teorema da adjacência. Assim, surge uma outra alternativa gráfica de visualização da tabela de verdade: os **mapas de Karnaugh**. Aqui o objetivo continua a ser identificar quando se pode aplicar o teorema da adjacência.

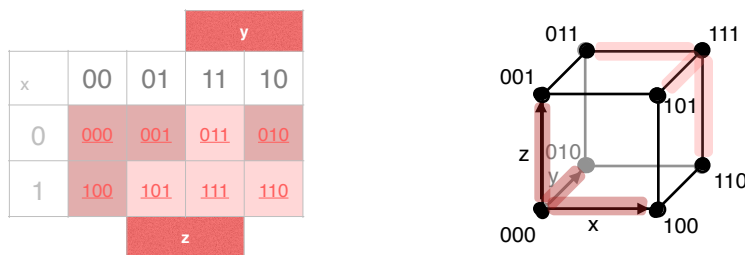
Os mapas de Karnaugh são “isomorfos” aos cubos booleanos. Tendo uma abordagem fundamentalmente heurística, este método é útil para  $n \leq 6$ .

Em cada dimensão, para garantir a adjacência, a evolução das coordenadas binárias de cada termo mínimo obedece ao código de Gray. Assim, apresentamos os mapas de Karnaugh para duas, três e quatro variáveis:



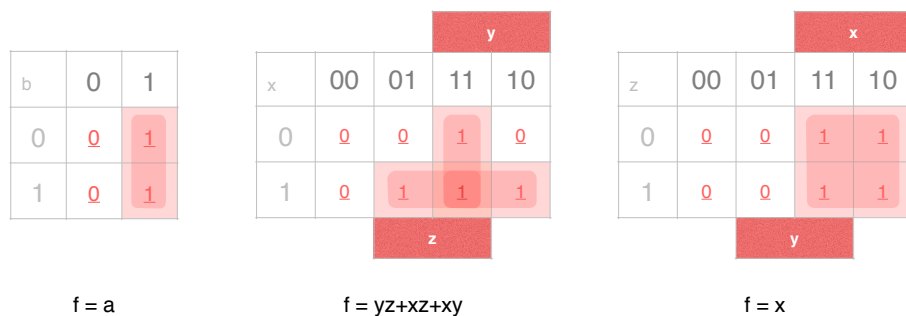
**figura 8**  
mapas de Karnaugh de 2, 3 e 4 variáveis

As adjacências num *K-map* (abreviatura de mapa de Karnaugh) podem-se ver como no seguinte exemplo, onde também podemos comprovar o isomorfismo de que falávamos atrás.



**figura 9**  
isomorfismo entre K-Map e cubo booleano

Vejamos, assim, alguns exemplos acompanhados das suas funções, com o K-map.



**figura 10**  
exemplos de aplicação de mapas de Karnaugh

Ao ter mapas com três variáveis, os seguintes exemplos ou seus semelhantes, podem acontecer.

|   |    |    |    |    |
|---|----|----|----|----|
|   |    |    | x  |    |
| z | 00 | 01 | 11 | 10 |
| 0 | 1  | 0  | 1  | 1  |
| 1 | 0  | 0  | 0  | 1  |
|   |    | y  |    |    |

$$f = \sum m(0,4,5,6) = \bar{z}\bar{y} + \bar{z}x + x\bar{y}$$

|   |    |    |    |    |
|---|----|----|----|----|
|   |    |    | x  |    |
| z | 00 | 01 | 11 | 10 |
| 0 | 1  | 0  | 1  | 1  |
| 1 | 1  | 0  | 1  | 1  |
|   |    | y  |    |    |

$$f = \sum m(0,1,4,5,6,7) = x + \bar{y}$$

figura 11

exemplos de aplicação de mapas de Karnaugh com três variáveis

Do mesmo modo, ao ter mapas com quatro variáveis, o seguinte exemplo ou seus semelhantes, podem acontecer.

|   |    |    |    |    |    |
|---|----|----|----|----|----|
|   |    |    |    | x  |    |
|   | zw | 00 | 01 | 11 | 10 |
|   | 00 | 1  | 0  | 0  | 1  |
|   | 01 | 0  | 1  | 0  | 0  |
| z | 11 | 1  | 1  | 1  | 1  |
|   | 10 | 1  | 1  | 1  | 1  |
|   |    |    | y  |    |    |

$$f = \sum m(0,2,3,5,6,7,8,10,11,14,15) = z + \bar{y}\bar{w} + \bar{x}yw$$

figura 12

exemplo de aplicação de mapas de Karnaugh com quatro variáveis

A minimização, tal como a segunda forma canónica, podem ser minimizados com maxtermos. Assim, teríamos, por exemplo:

|   |    |    |    |    |    |
|---|----|----|----|----|----|
|   |    |    |    | x  |    |
|   | zw | 00 | 01 | 11 | 10 |
|   | 00 | 1  | 0  | 0  | 1  |
|   | 01 | 0  | 1  | 0  | 0  |
| z | 11 | 1  | 1  | 1  | 1  |
|   | 10 | 1  | 1  | 1  | 1  |
|   |    |    | y  |    |    |

$$f = \sum m(0,2,3,5,6,7,8,10,11,14,15) = \prod M(1,4,9,12,13)$$

$$f = (\bar{x} + \bar{y} + z)(z + w + \bar{y})(z + \bar{w} + y)$$

$$\bar{f} = \sum m(1,4,9,12,13) = xy\bar{z} + \bar{z}\bar{w}y + \bar{z}w\bar{y}$$

$$\bar{f} = \overline{xy\bar{z} + \bar{z}\bar{w}y + \bar{z}w\bar{y}} = (\bar{x} + \bar{y} + z)(z + w + \bar{y})(z + \bar{w} + y)$$

figura 13

exemplo de aplicação de minimização com maxtermos





## Lógica de Polaridade

Tal como referimos no primeiro capítulo, o conceito lógico de 0's e 1's é apenas uma pura convenção, pois podia ser apenas verdadeiro/falso, maria/joão, ... Do mesmo modo, a convenção que temos de 1 ser correspondente ao nível elétrico mais positivo e o 0 ao nível elétrico mais baixo também é pura convenção. Além disso, existem duas convenções distintas, dependendo da realização eletrónica do equipamento em questão. Assim, podemos encontrar duas lógicas: a **lógica positiva** e a **lógica negativa**.

A descrição tabular do comportamento do circuito recorrendo aos níveis *high* e *low* (geralmente usados pelo seu acrónimo H e L, significam lógica positiva e lógica negativa, respetivamente) é inversa uma da outra. Em contextos não algébricos, a descrição funcional deve confinar-se aos símbolos H e L.

Por razões do contexto físico, interessa associar a um sinal uma determinada ação no circuito e o nível elétrico que o desencadeia: este pode ser H ou L (grande parte dos nossos equipamentos utilizam preferencialmente a L); o nível assim definido designa-se por **nível ativo**. Os nomes dos sinais para além de sugestivos devem mencionar também o nível ativo, como podemos ver alguns exemplos na tabela que se segue:

| active low | active high |
|------------|-------------|
| READY-     | READY+      |
| ERROR.L    | ERROR.H     |
| ADDR15(L)  | ADDR15(H)   |
| RESET*     | RESET       |
| ENABLE~    | ENABLE      |
| ~GO        | GO          |
| /RECEIVE   | RECEIVE     |
| TRANSMIT_L | TRANSMIT    |

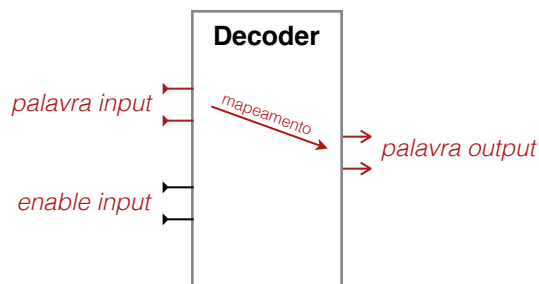
**lógica positiva**  
**lógica negativa**

**nível ativo**

**tabela 9**  
exemplos de nomes dos sinais

## Descodificadores (decoders)

Um **descodificador** é um circuito combinatório que permite, para uma palavra em código à entrada, ativar uma e uma só saída da função de descodificação. O descodificador mais comum de todos é o descodificador para o código binário natural. A **estrutura genérica** de um descodificador é a seguinte:



**descodificador**

**estrutura genérica**

**figura 17**  
estrutura genérica de um decodificador

Como podemos reparar temos um dado número de entradas para um dado número de saídas. Tipicamente esses números são  $n$  entradas (código) para  $2^n$  saídas, como são o caso os descodificadores 2:4, 3:8, 4:16, etc...

Na página seguinte podemos ver um descodificador 2 para 4 (2:4) e a sua tabela de verdade. Devemo-nos lembrar que o valor "x" corresponde a um *don't care*, valor a ser desprezado.

| inputs |    |    | outputs |    |    |    |
|--------|----|----|---------|----|----|----|
| EN     | I1 | I0 | Y3      | Y2 | Y1 | Y0 |
| 0      | x  | x  | 0       | 0  | 0  | 0  |
| 1      | 0  | 0  | 0       | 0  | 0  | 1  |
| 1      | 0  | 1  | 0       | 0  | 1  | 0  |
| 1      | 1  | 0  | 0       | 1  | 0  | 0  |
| 1      | 1  | 1  | 1       | 0  | 0  | 0  |

**figura 18**  
descodificador 2:4 e  
respetiva tabela de verdade

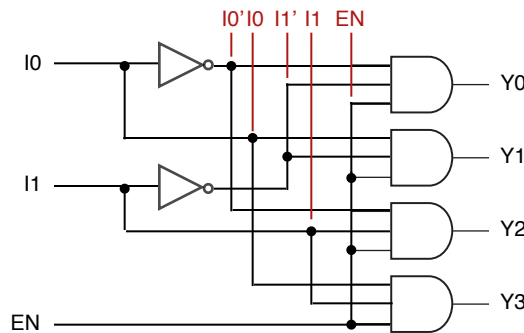
Como nos foi designada, esta é a tabela de verdade de um descodificador 2:4 e à esquerda encontra-se uma estrutura genérica do mesmo bloco combinatório. Mas qual é o seu diagrama lógico? E o que é a entrada EN?

A entrada EN, de nome **enable**, tal como o nome indica, pode ser vista como um interruptor, para nós, indicando qual a lógica que o sistema em questão usa. Neste caso, quando o EN está a zero, quaisquer que sejam os I1 o I0, o valor de Y0, Y1, Y2, Y3 é sempre 0, logo o nosso sistema é *active high*.

O **diagrama lógico** deste descodificador é o seguinte:

**enable**

**diagrama lógico**



**figura 19**  
diagrama lógico de um  
descodificador 2:4

É possível, de forma a construir um descodificador, por exemplo, 4:16, através de dois descodificadores 3:8, agrupando-os em **cascata**.

**cascata**

Podemos recorrer a um bloco de decodificação para obter os termos mínimos necessários à implementação de uma função booleana genérica.

## Codificadores (encoder)

Os **codificadores**, ao contrário dos descodificadores, têm como função principal, codificar uma dada palavra de código. Tendo tipicamente  $2^n$  entradas, codifica essas em palavras, com  $n$  saídas.

**codificadores**

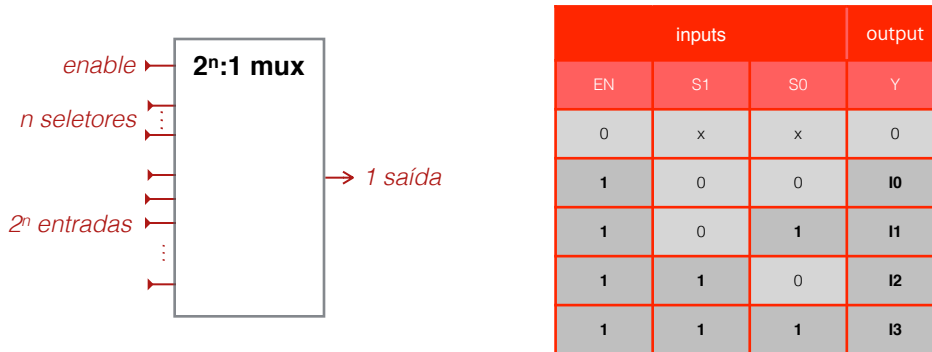
Há um grande problema com os codificadores - há que criar prioridade, de modo a evitar conflitos arbitrários no atendimento do código. Para tal existe um **codificador de prioridade**, o qual é um produto da redefinição da lógica interna através de variáveis intermédias  $H_n$ .

**codificador de prioridade**

Um codificador de prioridade conhecido é o 74x148, o qual tem lógica *active low*, *enable input*, *enable output* e *“got something”*.

## Multiplexers e demultiplexers

Enquanto os codificadores transmitem uma mensagem traduzida, os **multiplexers** (abreviatura *mux*) são dispositivos que selecionam uma de várias entradas e a faz seguir numa só saída. Um multiplexer de  $2^n$  entradas tem  $n$  linhas de seleção. Estes dispositivos são usados para aumentar a quantidade de dados a serem enviados numa rede num determinado intervalo de tempo e numa determinada largura de banda. Vejamos, assim, o seguinte multiplexer genérico, juntamente com uma tabela de verdade de um multiplexer 4:1.



**figura 20**  
multiplexer genérico e  
tabela de verdade de um  
multiplexer 2:1

A função booleana que parte destes dispositivos é muito própria. Um multiplexer 2:1 tem a seguinte expressão:

$$Y = (I_0\bar{S}) + (I_1S)$$

**equação 42**  
equação de um mux 2:1

De uma forma mais geral, um *mux* tem como expressão:

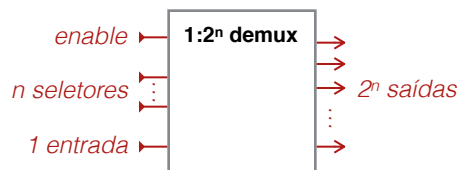
$$Y = EN \cdot \left( \sum_k m_k(S) I_k \right) \quad k = 0, \dots, 2^n - 1$$

**equação 43**  
equação geral de um mux  
2^n:1

onde  $m_k(S)$  é o k-ésimo mintermo nas variáveis de seleção  $S_0, S_1, \dots, S_n$ .

Por sua vez, os **demultiplexers** têm uma função inversa aos *mux*'s. Estes, ao invés de partirem de  $2^n$  entradas, partem de uma só, abrindo para  $2^n$  saídas.

**demultiplexers**



**figura 21**  
demultiplexer genérico

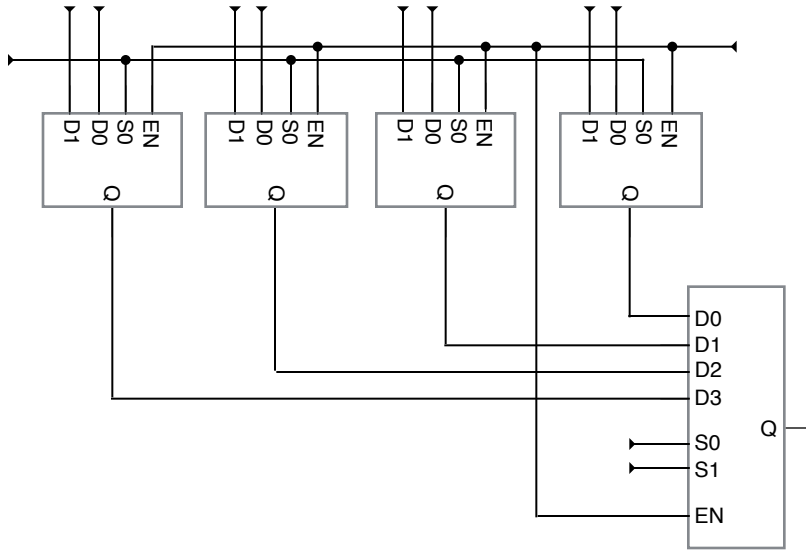
A equação geral de um *demux* - abreviatura de demultiplexer - é a seguinte, considerando  $O_k$  como as nossas saídas e  $D_{in}$  como entrada:

$$O_k = EN \cdot D_{in} \cdot m_k(S) \quad k = 0, \dots, 2^n - 1$$

**equação 44**  
equação geral de um demux  
1:2^n  
**hierarquizar**

É possível **hierarquizar** multiplexers. A chamada hierarquia de multiplexagem existe muitas vezes por causa da falta de maiores multiplexers. Assim, conseguimos,

por exemplo, criar um *mux* 8:1 a partir de quatro 2:1 e um 4:1, como podemos ver a seguir:



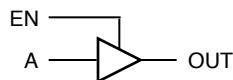
**figura 22**  
mux 8:1 a partir de quatro  
mux's 2:1 e um 4:1

## Buffers 3-state

Um multiplexer pode ser construído através de **buffers tri-state** - dispositivos lógicos que exibem três estados possíveis. Dois dos estados que estes podem criar já são usados: a lógica 1 e a lógica 0. O terceiro estado, porém, é denominado de **alta-impedância** (Hi-Z) e é neste que a lógica se comporta como um circuito aberto, isto é, parecendo que a saída está, de certo modo, desligada, como se não tivesse significado lógico. Um buffer tri-state pode ser usado ao invés das portas AND e NAND, sendo que é mais preferida, até mesmo por motivos económicos.

**buffers tri-state**

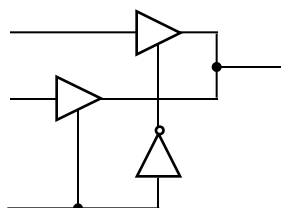
**alta-impedância**



| inputs |   | output |
|--------|---|--------|
| EN     | A | OUT    |
| L      | L | Hi-Z   |
| L      | H | Hi-Z   |
| H      | L | L      |
| H      | H | H      |

**figura 23**  
buffer tri-state e respetiva  
tabela de verdade

O conceito de alta-impedância pode ser traduzido, por enquanto, como uma resistência praticamente infinita, sem ligação. Através de buffers tri-state também podemos criar estratégias diferentes de multiplexagem. Vejamos este exemplo de multiplexagem revisitada, num multiplexer 2:1.



**figura 24**  
multiplexagem revisitada por  
buffers tri-state

## 7. Circuitos Aritméticos

Atrás, na unidade “Sistemas Numéricos e Códigos” (ver §Sistemas Numéricos e Códigos), estudámos como efetuar cálculos algébricos básicos com notação binária. Agora, e com os conhecimentos adquiridos nas unidades seguintes, podemos aprender como efetuar os mesmos cálculos com o auxílio de dispositivos lógicos, como blocos combinatórios. Damos assim ênfase aos **circuitos aritméticos**.

**circuitos aritméticos**

### Circuitos somadores

Se nos fosse pedido para representar o número  $-17_{10}$  em sinal e módulo, complemento para 1 e complemento para 2, nós seríamos capazes de o fazer. Sabemos pois, de unidades anteriores que números em complemento para um podem ser adicionados aplicando regras habituais de adição binária, nos quais os transportes para além do bit mais significativo devem ser somados ao resultado (para evitar que o zero seja contado duas vezes). Sabemos também que números em complemento para dois podem ser adicionados aplicando regras habituais de adição binária, ignorando transportes para além do bit mais significativo. Já em sinal e módulo é bastante difícil construir um circuito digital que some dois números, dado que é necessário comparar as magnitudes dos operandos para determinar o sinal do resultado. O *overflow* acontece se a soma de dois números positivos produzir um número negativo, ou se a soma de dois números negativos produzir um resultado positivo. Em complemento para dois, o *overflow* acontece se no bit mais significativo  $c_{in} \neq c_{out}$ .

Para somar e subtrair números em complemento para dois, precisamos de um **circuito somador**.

Um **half-adder**<sup>5</sup> (em português semi-somador) é um circuito que soma dois operandos de um bit cada, e produz um resultado de dois bits, que varia entre 0 e 2.

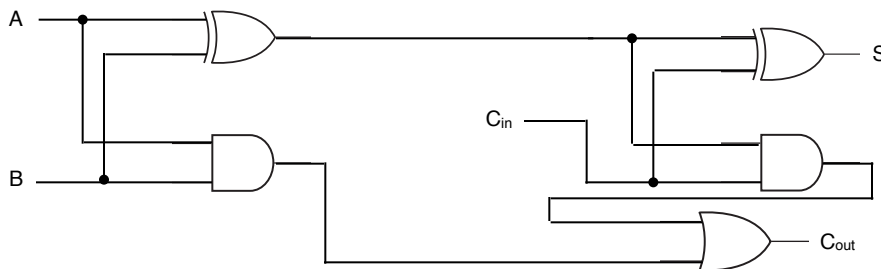
**circuito somador  
half-adder**



**figura 25**  
half-adder e respetiva  
tabela de verdade

Para somar operandos com mais que um bit temos que assegurar a transferência de transportes (*carries*) entre vários bits. Para tal, necessitamos de um somador multi-bit, construído à custa de **somadores completos** (em inglês *full adders*).

**somadores completos**



**figura 26**  
somador completo

<sup>5</sup> A porta que forma a saída HS é uma porta lógica XOR (*exclusive-or*), pelo que é representada pelo símbolo  $\oplus$ .

Este circuito somador tem as seguintes equações:

$$C_{out} = A \cdot B + A \cdot C_{in} + B \cdot C_{in}$$

$$S = A \oplus B \oplus C_{in}$$

equação 45

equações de saída de um somador completo

Dois operandos binários de  $n$  bits podem ser somados com uma cascata de  $n$  somadores completos cada um dos quais um bit do resultado. A esta cascata de somadores completos damos o nome de **ripple adder**. A figura abaixo representa um ripple adder de quatro bits. O bit de transporte para o bit menos significativo ( $c_0$ ) é iniciado com o valor 0, e o bit de transporte (saída) para o seguinte somador está conectado ao bit de transporte (entrada), do bit de seguinte significado.

ripple adder

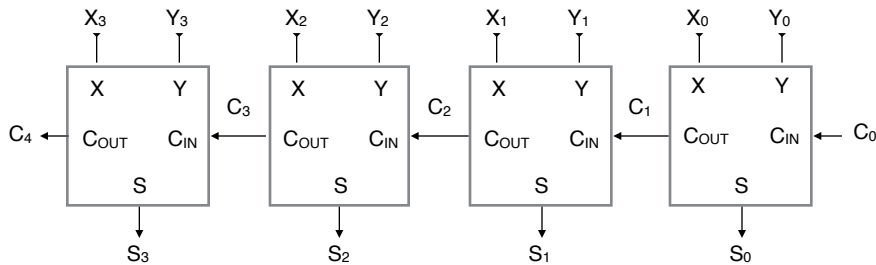


figura 27

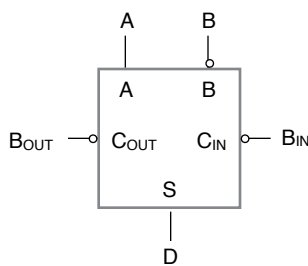
ripple adder

O ripple adder é um **circuito iterativo** clássico. Um circuito iterativo é um circuito que se usa para calcular um resultado de  $n$  bits, com  $n$  módulos interligados em cascata, em que cada um dos módulos “seguintes” faz cálculos com base nos resultados produzidos pelo módulo “anterior”.

circuito iterativo

Para subtrair operandos com mais que um bit, temos que assegurar a transferência de transportes (*borrows*) entre vários bits. Tal subtrator multi-bit pode ser construído à custa de **subtratores completos**.

subtratores completos



| inputs          |   |   | output           |   |
|-----------------|---|---|------------------|---|
| B <sub>IN</sub> | A | B | B <sub>OUT</sub> | D |
| 0               | 0 | 0 | 0                | 0 |
| 0               | 0 | 1 | 1                | 1 |
| 0               | 1 | 0 | 0                | 1 |
| 0               | 1 | 1 | 0                | 0 |
| 1               | 0 | 0 | 1                | 1 |
| 1               | 0 | 1 | 1                | 0 |
| 1               | 1 | 0 | 0                | 0 |
| 1               | 1 | 1 | 1                | 1 |

figura 28

subtrator completo e respetiva tabela

As equações de saída são as seguintes:

$$B_{OUT} = \bar{A}B + \bar{A}B_{IN} + BB_{IN}$$

$$D = A \oplus B \oplus B_{IN}$$

equação 46

equações de saída de um subtrator completo

Um somador completo pode realizar a operação de subtração  $a - b$ , fazendo o cálculo de  $a + 1sc(b) + 1$  (isto dado que  $-b = 1sc(b) + 1$ ).

$$\begin{aligned}C_{OUT} &= AB + AC_{IN} + BC_{IN} \\S &= A \oplus B \oplus C_{IN} \\ \bar{B}_{OUT} &= (A + \bar{B})(A + \bar{B}_{IN})(\bar{B} + \bar{B}_{IN}) = A\bar{B} + A\bar{B}_{IN} + \bar{B}\bar{B}_{IN} \\D &= A \oplus \bar{B} \oplus \bar{B}_{IN}\end{aligned}$$

**equação 47**  
operações de subtração por um somador completo

Também podemos criar a operação de subtração em cascata. Para tal, com um ripple adder, basta fornecer as entradas  $A$  e  $B'$  e o bit de transporte no bit menos significativo com o valor 1.

## Somadores carry-lookahead

A equação lógica para um bit de soma  $i$  de um somador binário pode ser simplesmente expressa por:

$$\begin{aligned}S_i &= A_i \oplus B_i \oplus C_i \\C_{i+1} &= A_i B_i + A_i C_i + B_i C_i\end{aligned}$$

**equação 48**  
expressão lógica para um bit de soma  $i$  de um somador binário gerado

Na fase  $i$  é **gerado** um *carry* se para alguma combinação de  $A_i$  e  $B_i$  é produzido  $C_{i+1}=1$ , independentemente das entradas  $a_0, \dots, a_{i-1}, b_0, \dots, b_{i-1}$  e  $c_0$ . Denominemos  $g$  como o sinal de geração de *carry*. Assim,

$$g_i = A_i B_i$$

**equação 49**  
expressão da geração

Também na fase  $i$  é **propagado** um *carry* se é produzido  $C_{i+1}=1$  na presença de tal combinação de entradas  $a_0, \dots, a_{i-1}, b_0, \dots, b_{i-1}$  e  $c_0$  que causam  $c_i=1$ . Denominemos  $p$  como o sinal de propagação de *carry*. Assim,

$$p_i = A_i + B_i$$

**propagado**

**equação 50**  
expressão da propagação

Por substituição, temos que:

$$C_{i+1} = g_i + p_i c_i$$

**equação 51**  
expressão final por substituição

Isto é, uma fase produz um transporte (*carry*) se gera um transporte ou se propaga um *carry* e o bit de transporte tem valor atual 1. De forma a eliminar um provável transporte iterativo nós expandimos o termo  $c_i$  para cada fase  $i$  e multiplicamo-lo para obter uma expressão AND-OR de segundo nível. Usando esta técnica, podemos obter as seguintes quatro fases de adição:

$$\begin{aligned}C_1 &= g_0 + p_0 c_0 \\C_2 &= g_1 + p_1 c_1 = g_1 + p_1 (g_0 + p_0 c_0) = g_1 + p_1 g_0 + p_1 p_0 c_0 \\C_3 &= g_2 + p_2 c_2 = g_2 + p_2 (g_1 + p_1 g_0 + p_1 p_0 c_0) = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0 \\C_4 &= g_3 + p_3 c_3 = g_3 + p_3 (g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0) = \\&= g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0\end{aligned}$$

**equação 52**  
expressão das quatro fases de adição

## Somadores BCD

Se somarmos dois dígitos BCD (e *carry*) podemos obter uma soma que varia entre 0 e 19. Vejamos a seguinte tabela:

| soma | notação binária  |      | código BCD       |      |
|------|------------------|------|------------------|------|
|      | <i>carry out</i> | soma | <i>carry out</i> | soma |
| 0    | 0                | 0000 | 0                | 0000 |
| 1    | 0                | 0001 | 0                | 0001 |
| 2    | 0                | 0010 | 0                | 0010 |
| 3    | 0                | 0011 | 0                | 0011 |
| 4    | 0                | 0100 | 0                | 0100 |
| 5    | 0                | 0101 | 0                | 0101 |
| 6    | 0                | 0110 | 0                | 0110 |
| 7    | 0                | 0111 | 0                | 0111 |
| 8    | 0                | 1000 | 0                | 1000 |
| 9    | 0                | 1001 | 0                | 1001 |
| 10   | 0                | 1010 | 1                | 0000 |
| 11   | 0                | 1011 | 1                | 0001 |
| 12   | 0                | 1100 | 1                | 0010 |
| 13   | 0                | 1101 | 1                | 0011 |
| 14   | 0                | 1110 | 1                | 0100 |
| 15   | 0                | 1111 | 1                | 0101 |
| 16   | 1                | 0000 | 1                | 0110 |
| 17   | 1                | 0001 | 1                | 0111 |
| 18   | 1                | 0010 | 1                | 1000 |
| 19   | 1                | 0011 | 1                | 1001 |

**tabela 10**

soma por notação binária e por BCD

Como podemos ver, no caso da notação binária, a partir do número 16, existe *carry*, pelo que, para quatro bits, se compararmos com o código BCD temos que somar a quantidade 6 a partir do dígito 9 (exclusivé), de forma a fornecer correção perante o cálculo.

## Unidade aritmética e lógica (ALU)

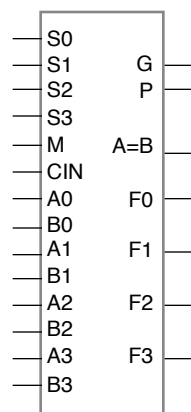
Uma **unidade aritmética e lógica** é um dispositivo combinatório que executa qualquer operação aritmética ou lógica (de um conjunto predefinido), sobre um par de operandos de  $n$  bits. A operação a executar é especificada com entradas de seleção de função.

**unidade aritmética e lógica**

| S3 | S2 | S1 | S0 | M=0 (operação aritmética)                                | M=1 (operação lógica)    |
|----|----|----|----|----------------------------------------------------------|--------------------------|
| 0  | 0  | 0  | 0  | $F = A - 1 + \text{CIN}$                                 | $F = A'$                 |
| 0  | 0  | 0  | 1  | $F = A \text{ AND } B - 1 + \text{CIN}$                  | $F = A' \text{ OR } B'$  |
| 0  | 0  | 1  | 0  | $F = A \text{ AND } B' - 1 + \text{CIN}$                 | $F = A' \text{ OR } B$   |
| 0  | 0  | 1  | 1  | $F = 1111 + \text{CIN}$                                  | $F = 1111$               |
| 0  | 1  | 0  | 0  | $F = A + (A \text{ OR } B) + \text{CIN}$                 | $F = A' \text{ AND } B'$ |
| 0  | 1  | 0  | 1  | $F = A \text{ AND } B + (A \text{ OR } B) + \text{CIN}$  | $F = B'$                 |
| 0  | 1  | 1  | 0  | $F = A - B - 1 + \text{CIN}$                             | $F = A \text{ XOR } B'$  |
| 0  | 1  | 1  | 1  | $F = A \text{ OR } B' + \text{CIN}$                      | $F = A \text{ OR } B'$   |
| 1  | 0  | 0  | 0  | $F = A + (A \text{ OR } B) + \text{CIN}$                 | $F = A' \text{ AND } B$  |
| 1  | 0  | 0  | 1  | $F = A + B + \text{CIN}$                                 | $F = A \text{ XOR } B$   |
| 1  | 0  | 1  | 0  | $F = A \text{ AND } B' + (A \text{ OR } B) + \text{CIN}$ | $F = B$                  |
| 1  | 0  | 1  | 1  | $F = A \text{ OR } B + \text{CIN}$                       | $F = A \text{ OR } B$    |
| 1  | 1  | 0  | 0  | $F = A + A + \text{CIN}$                                 | $F = 0000$               |
| 1  | 1  | 0  | 1  | $F = A \text{ AND } B + A + \text{CIN}$                  | $F = A \text{ AND } B'$  |
| 1  | 1  | 1  | 0  | $F = A \text{ AND } B' + A + \text{CIN}$                 | $F = A \text{ AND } B$   |
| 1  | 1  | 1  | 1  | $F = A + \text{CIN}$                                     | $F = A$                  |

**tabela 11**  
operações num ALU

Na figura abaixo podemos ver o circuito ALU (componente 74x181), onde podemos criar correspondência à tabela acima.



**figura 29**  
circuito ALU

## Multiplicação de números binários

Para estudarmos a multiplicação de números binários devemos subdividir esta secção em duas partes: multiplicação de números sem sinal e multiplicação de números com sinal. Começemos pela primeira.

Numa multiplicação de números binários sem sinal, os processos de multiplicação obedecem às mesmas regras básicas existentes no sistema decimal. Vejamos o seguinte exemplo:

$$12 \times 13 = 156$$

$$\begin{array}{r}
 1101 \quad \leftarrow \text{multiplicando} \\
 \times 1101 \quad \leftarrow \text{multiplicador} \\
 \hline
 1101 \\
 0000 \\
 1100 \quad \leftarrow \text{produtos parciais} \\
 + 1100 \\
 \hline
 10011100
 \end{array}$$

figura 30

multiplicação binária sem sinal

Na multiplicação com sinal deve-se, a cada passo, fazer a extensão do sinal. Se o multiplicador for negativo, a última cópia do multiplicando deve ser negada. Assim, por exemplo:

$$-5 \times 3 = -15$$

$$-5 \times -3 = +15$$

$$\begin{array}{r}
 1011 \\
 \times 1101 \\
 \hline
 00000 \\
 10011 \\
 \hline
 111011 \\
 11011 \\
 \hline
 1110001 \\
 00000 \\
 \hline
 11110001 \\
 00000 \\
 \hline
 11110001
 \end{array}
 \qquad
 \begin{array}{r}
 1011 \\
 \times 1101 \\
 \hline
 00000 \\
 11011 \\
 \hline
 111011 \\
 00000 \\
 \hline
 1111011 \\
 11011 \\
 \hline
 11100111 \\
 00101 \\
 \hline
 00001111
 \end{array}$$

figura 31

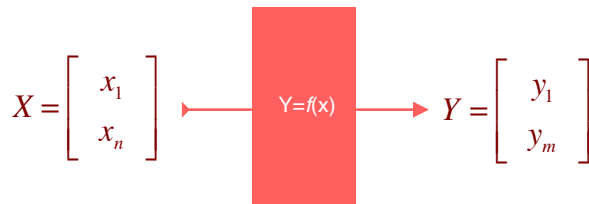
multiplicação binária com sinal (2 exemplos)

## 8. Sistemas Sequenciais

Como pudemos estudar na primeira unidade (ver §Ambiente digital), existem dois tipos genéricos de circuitos: circuitos combinatórios e circuitos sequenciais. Estudados os circuitos combinatórios, passamos agora para os sequenciais, com uma abordagem algo semelhante com a dada até ao momento, embora o ambiente com que trabalhamos agora, mude em grande parte.

### Lógica sequencial versus lógica combinatória

Atrás foi referido que um sistema combinatório é um sistema que faz corresponder diretamente uma entrada simbólica  $x_n$  a uma saída simbólica  $y_m$ , tal como podemos ver na figura abaixo.



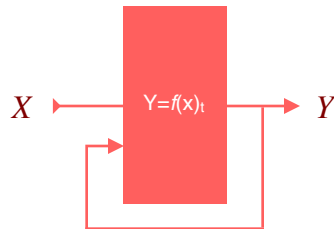
**figura 32**  
um sistema combinatório

Um circuito combinatório, como sabemos, então, é um sistema que não possui realimentação, pois nenhuma saída serve de re-entrada do sistema, dado que apenas depende da entrada atual. Num **sistema sequencial** temos uma situação contrária. Na primeira unidade (ver §Ambiente digital, página 5) temos um circuito sequencial. Como podemos ver através dessa figura, existe **realimentação** (geralmente denominada pelo termo em inglês *feedback*), pelo que a saída depende das entradas atuais, mas também da informação passada. De uma forma muito genérica, temos perante nós, uma **memória**. A análise de um sistema deste calibre conjuga-se, assim, com a sua variação no tempo.

**sistema sequencial**

**realimentação**

**memória**

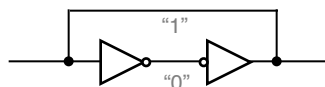


**figura 33**  
um sistema sequencial

### Estruturas elementares de memória

Havendo feedback existe salvaguarda de informação por dados intervalos de tempo até um evento como uma perturbação do sistema, o que pode provocar a alteração do valor lógico atual.

A memória mais simples que se pode criar ou desenhar são inversores em cascata, os quais constituem uma célula de memória estática.



**figura 34**  
célula de memória estática

Com este tipo de memória encontramos-nos com um grande problema: então como é que escrevemos informação? Uma possível solução é a quebra seletiva da realimentação, instalando interruptores.

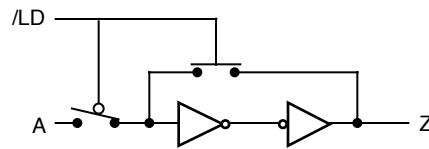


figura 34

quebra seletiva do feedback

Geralmente, a solução mais apresentada é um outro circuito, outro desenho: um **bi-estável**. Pela sua fácil aplicação e desenho, o bi-estável é uma forma muito usual de apresentação da cascata de inversores.

bi-estável

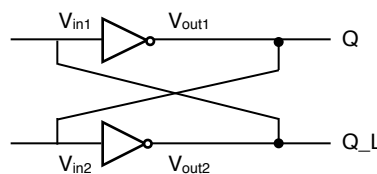


figura 34

circuito bi-estável

Em regime estável, a variável  $Q$  designa-se como variável de estado de elemento de memória. Se atribuirmos valores aleatórios para o arranque de uma simulação, iremos constatar que o circuito apresenta dois estados possíveis e que os alterna num intervalo de tempo específico. Mas entre os dois estados possíveis, à transição, damos o nome de **meta-estabilidade**, dado que é um “estado” instável da máquina, desregulado, na maior parte das vezes. Quando uma máquina se encontra neste ponto, isto significa que o estado seguinte é imprevisível e que depende de estímulos aleatórios como o ruído elétrico.

meta-estabilidade

## Latch SR

Como vimos anteriormente, a uma cascata de inversores demos o nome de memória. Mais especificamente, o primeiro exemplo trata-se de uma *static latch*. Uma latch é um circuito que permite que dados sejam guardados e que hajam várias transições durante um dado intervalo de tempo.

latch

Existem vários tipos de latches. Uma **latch SR** é um circuito sequencial, numa cascata de portas lógicas NOR (portas OR com inversores). O diagrama lógico de uma latch SR é o seguinte:

latch SR

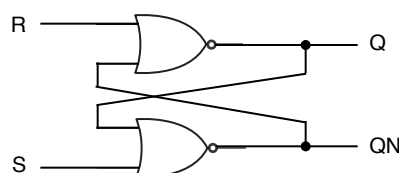


figura 35

diagrama lógico de uma latch SR

Para representar uma latch SR basta desenhar o seu bloco.

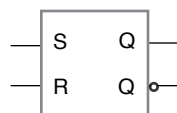


figura 36

bloco lógico de uma latch SR

As variáveis de controlo R e S têm estas designações em especial já vemos porquê. Se ambas R e S tiverem o valor L (do inglês *low* - baixo) ( $R = S = L$ ), as portas NOR ficarão ativas e, até que outra entrada seja referida, cada porta é simplesmente um inversor tal como requerido para uma latch. Assim com  $R = S = L$  a máquina pode encontrar-se em dois estados possíveis, pelo que o em que se encontra vai depender do estado passado.

Suponhamos agora que a entrada R adquire o valor H (do inglês *high* - alto). Assim, a porta NOR que lhe segue permanece inativa, por conseguinte  $Q = L$ . Isto é, o facto de que o R se ter tornado H fez um *reset* à latch. É por esta razão que a entrada R foi etiquetada para R, de **reset**. Se a latch se iniciou no estado reset, atribuindo o valor H a R não produziria qualquer variação de estado.

Até aqui temos que se  $R = S = L$  qualquer um dos estados H ou L é possível, e que se  $R = H$  permanente ou temporariamente, a latch estará em reset. De forma algo análoga, iniciando a máquina com o valor  $R = S = L$ , se S (etiquetado de *set*) mudar o seu valor para H, permanente ou temporariamente, a latch irá para um estado set ou ficará no estado set, se nele já ela se encontrar.

O comportamento de uma latch pode ser convenientemente sumariado numa tabela de verdade apropriadamente arranjada para acolher estados de transição. A esta tabela específica damos o nome de **tabela de transição**. A tabela de transição de uma latch SR é a seguinte:

| inputs |   | outputs  |          |
|--------|---|----------|----------|
| S      | R | Q        | QN       |
| L      | L | last Q   | last QN  |
| L      | H | L        | H        |
| H      | L | H        | L        |
| H      | H | not used | not used |

reset

tabela de transição

tabela 12

tabela de transição de uma latch SR

Se simularmos  $S = R = H$  podemos constatar que os complementos de Q igualam-se, o que é impossível ( $Q = Q'$ ), pelo que na tabela se encontra a notação “not used”.

De forma a concretizar a análise de um sistema sequencial como este, é hábito, e é útil, fazer um **diagrama de estados**. Neste caso, e de acordo com a tabela de transições em cima, podemos criar o seguinte diagrama de estados:

diagrama de estados

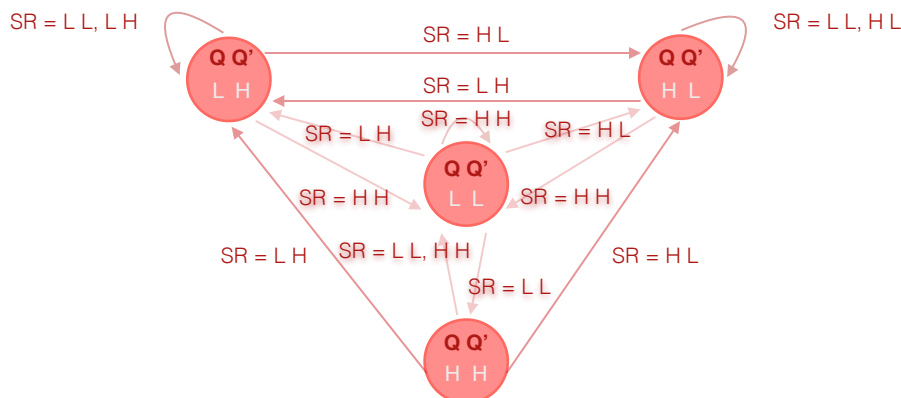


figura 37

diagrama de estados de uma latch SR

Mas este diagrama de estados contém um estado que está indicado como “not used”. Quando tal acontece, podemos ocultá-lo, dado que num diagrama mais real, só se a máquina se iniciasse nesse estado é que tal poderia existir. Assim, não há mal nenhum em apresentar um diagrama como o seguinte:

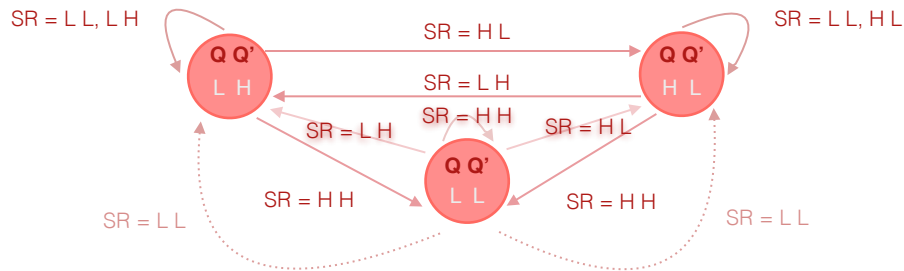


figura 38

diagrama de estados de uma latch SR sem “not used”

**Nota!** Note-se bem que a figura contém um retorno ambíguo ao estado L-H ou H-L.

nota

Como um circuito sequencial é um circuito que vive de realimentação (feedback), a evolução do sistema ao longo do tempo é um aspeto importante a considerar. Assim, existem certos **parâmetros temporais**, entre os quais, tempos de propagação e largura mínima de pulso. Vejamos a seguinte imagem.

parâmetros temporais

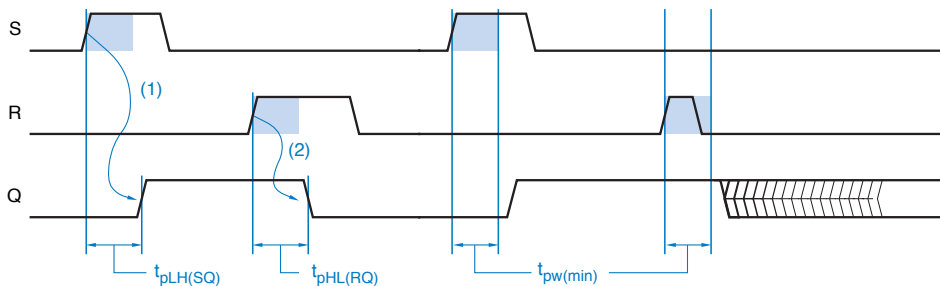


figura 39

diagrama temporal de uma latch SR

A figura acima define parâmetros temporais para uma latch SR. O **tempo de propagação** é o intervalo de tempo entre a perturbação dada na entrada até que seja efetuada alteração na saída. Embora nos possa parecer instantâneo, como a ordem de grandeza é do nanossegundo, é importante saber que existem atrasos nestes sistemas. Os tempos de propagação diferem de equipamento para equipamento, tal como o tempo de propagação de high para low pode ser diferente da de low para high. Com uma latch SR uma transição low para high em S pode causar uma transição igual em Q, o que provoca um tempo de propagação próprio, tal como a figura nos indica. De forma semelhante, uma transição de low para high em R pode causar uma transição de high para low em Q, com o tempo de propagação expresso por  $t_{pHL(RQ)}$ , como se pode averiguar na segunda transição na figura.

tempo de propagação

A **largura mínima de pulso** é uma especificação que é usualmente dada para as entradas S e R da latch. Tal como se pode ver na figura acima, a latch pode ir para um meta-estado e permanecer assim durante algum tempo, aleatório, se um pulso mais pequeno que a largura mínima de pulso  $t_{pw(min)}$  for aplicada em S ou em R. A latch pode ir para meta-estado sempre que tal situação ocorrer.

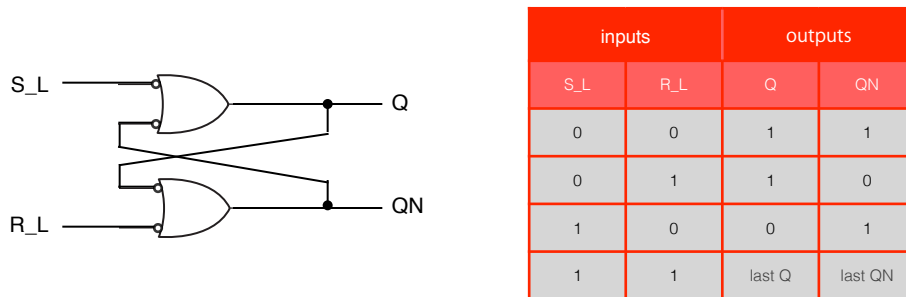
largura mínima de pulso

## Latch SR com portas NAND

Uma latch SR com portas NAND (em inglês muitas vezes dita como *S-bar-R-bar latch*) com lógica active-low em S e R, pode ser construída através de portas

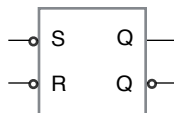
# 43 INTRODUÇÃO AOS SISTEMAS DIGITAIS

NAND tal como podemos ver na figura seguinte, tal como a tua tabela de estados de transição. Podemos assim ver que existem muitas semelhanças entre uma latch SR por NOR's e uma latch SR com NAND's.



**figura 40**  
latch SR com portas NAND  
e respetiva tabela de  
transição

Primeiro, S\_L e R\_L estão em lógica active-low, logo a latch recorda o seu estado passado quando  $S_L = R_L = 1$ ; as entradas active-low estão claramente indicadas na representação em bloco lógico, que se segue.

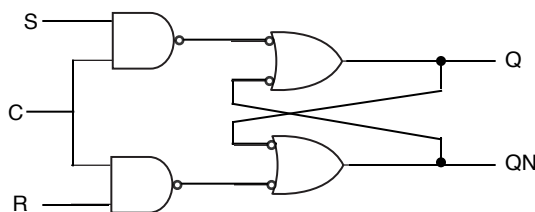


**figura 41**  
bloco lógico de uma latch SR  
com portas NAND

Segundo, quando ambos S\_L e R\_L são alterados em simultâneo ambos as saídas vão para 1, e não para 0, como acontecia na latch SR. À exceção destas diferenças, a operação desta latch é igual à da latch SR, incluindo situações de meta-estabilidade e tempos.

## Latch SR com enable

Uma latch SR ou uma latch S'R' é sensível às entradas S e R, sempre. No entanto, e como forma de facilitar a alternância dos valores das entradas, podemos adicionar um enable C.



**figura 42**  
diagrama lógico de uma  
latch SR com enable

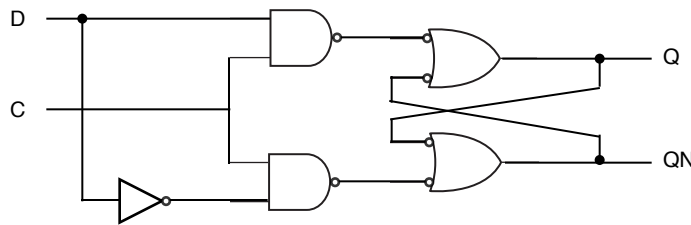
## Latch D

As latches SR são úteis em termos de controlo de dadas aplicações. Assim, nós controlamos através do valor que atribuímos às variáveis R e S.

A figura seguinte (na página seguinte) apresenta uma **latch D**. O seu diagrama lógico é passível de ser reconhecido a partir de uma latch SR com enable, com apenas um inversor adicionado de forma a gerar S e R através de D (entrada com o nome D de dados - *data* em inglês). Isto elimina o problema que tínhamos em latches SR, onde S e R podiam ser alterados em simultâneo. A variável de controlo de entrada de uma latch D é denominada de C, mas também pode ter os nomes de ENABLE, CLK e G.

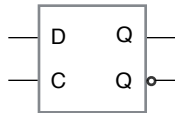
**latch D**

A lógica deste componente é active-low, pelo que também tem uma largura mínima de pulso, como requisito de alternância.



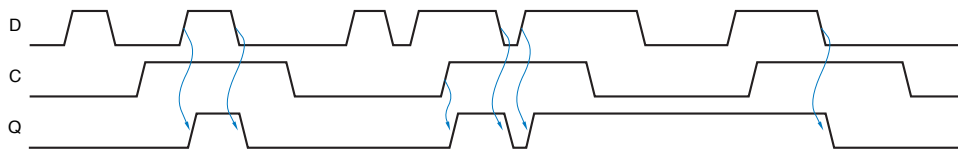
**figura 43**  
diagrama lógico de uma latch D

A representação de bloco lógico de um componente desta espécie é a seguinte:



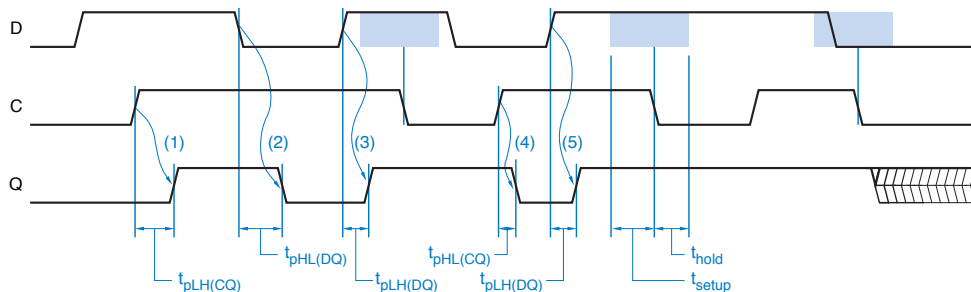
**figura 44**  
bloco lógico de uma latch D

Um exemplo de comportamento de uma latch D está dada na figura seguinte. Quando C é alterado, a saída Q segue D. Nesta situação, o latch diz-se “aberto” e o caminho de D até Q é “transparente”; o circuito também é chamado de *transparent latch* por esta razão em particular. Quando o C está negado, o latch “fecha”; a saída Q fixa o seu último valor e enquanto C permanece negado não há alteração em resposta a D.



**figura 45**  
diagrama temporal de uma latch D

De forma mais detalhada, a seguinte figura incorpora os tempos de atraso adjacentes ao componente. Quatro diferentes tempos de atraso são mostrados na figura, para sinais que se propagam das entradas C e D à saída Q. Por exemplo, nas transições (1) e (4) a latch é inicialmente “fechada” e a entrada D é o oposto da saída Q. Portanto, quando C altera o seu valor para 1 a latch “abre-se” e a saída Q muda após o atraso  $t_{pLH(CQ)}$  ou  $t_{pHL(CQ)}$ .



**figura 46**  
diagrama temporal de uma latch D (em detalhe)

Embora a latch D elimine o problema da latch SR de  $S = R = 1$ , ela não elimina o problema da meta-estabilidade. Tal como presente na figura acima, há uma zona sombreada (a terceira) onde a entrada D não deve mudar o seu valor, dado que deve respeitar os tempos de **setup** - tempo mínimo antes do clock, dentro do qual as

**tempo de setup**

entradas devem permanecer estáveis - e de **hold** - tempo mínimo depois do clock, no qual as entradas devem permanecer estáveis -, caso contrário temos situações de meta-estabilidade, evitáveis nestes casos.

Uma latch D tem a seguinte tabela de estados:

| inputs |   | outputs |         |
|--------|---|---------|---------|
| C      | D | Q       | QN      |
| 0      | x | last Q  | last QN |
| 1      | 0 | 0       | 1       |
| 1      | 1 | 1       | 0       |

**tabela 13**

tabela de transição de uma latch D

## Flip-flops

Nas secções anteriores referimos sempre latches, como circuitos sequenciais. Mais que as latches, os flip-flops são o próximo e último exemplo de circuitos sequenciais que vamos aprender nesta disciplina. Mas qual é a diferença entre ambos? Os latches são sistemas que analisam as suas entradas continuamente e que mudam as suas saídas em qualquer instante, independentemente de qualquer sinal de relógio (clock). Já os **flip-flops** são sistemas sequenciais que analisam, normalmente, as suas entradas e que mudam as suas saídas apenas em instantes determinados por um sinal de relógio.

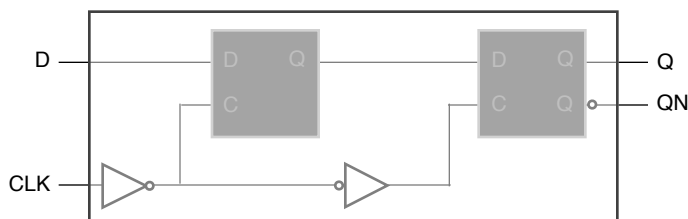
## Flip-flop D edge-triggered

Um **flip-flop D edge-triggered** combina um par de latches D, tal como mostrado na figura seguinte, de forma a criar um circuito que copia a sua entrada D e muda as duas saídas Q e QN apenas quando há uma transição low para high no clock (edge-triggered). O primeiro latch é denominado de **master**. Este latch é sempre aberto enquanto CLK é 0. Quando a entrada de relógio CLK muda o seu valor para 1, a latch master é fechado e a sua saída é transferida para a segunda latch, denominada de **slave**. A latch slave está aberta enquanto o CLK está com valor 1, mas muda apenas no início deste intervalo, porque a master está fechada e incapacitada de mudar até ao resto do intervalo.

**flip-flop D edge-triggered**

**master**

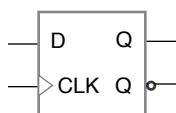
**slave**



**figura 47**

diagrama lógico interno de um flip-flop D edge-triggered

A representação em bloco lógico é a seguinte:

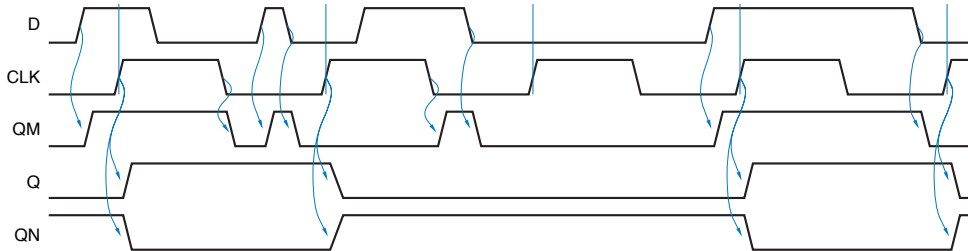


**figura 48**

bloco lógico de um flip-flop D edge-triggered

O triângulo na entrada CLK, no flip-flop D edge-triggered, indica que existe comportamento edge-triggered, ou seja, que ele reage à transição low para high e é

denominado de *dynamic-input indicator* (em português, indicador de entrada dinâmica). A figura seguinte mostra o comportamento funcional deste flip-flop num diagrama temporal. Note-se que na figura o sinal QM corresponde à saída da latch master. Notemos também que a variável QM muda apenas quando CLK está a 0. Quando CLK muda para 1, o valor atual de QM muda para Q, e QM permanece assegurado em relação a uma possível mudança até que o CLK retorne a 0.

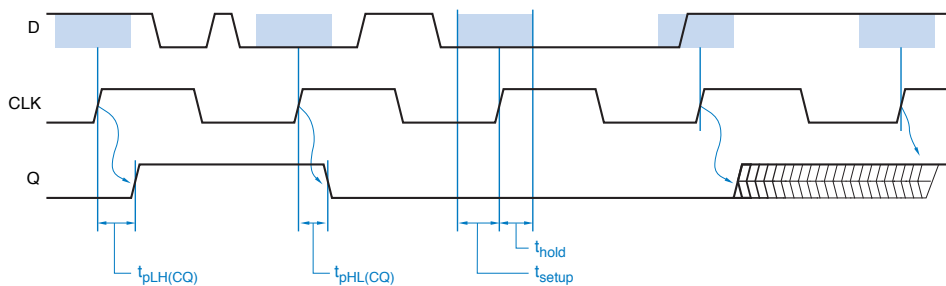


**figura 49**  
diagrama temporal de um  
flip-flop D edge-triggered

A figura acima mostra o comportamento do flip-flop, ao longo do tempo, mais a pormenor. Todos os tempos de atraso são medidos a partir do início de uma transição low para high do CLK, dado que esse é o único acontecimento que causa uma mudança de estado de saída.

Tal como uma latch D, um flip-flop D edge-triggered também tem os seus tempos de setup e hold, durante os quais as suas entradas não devem mudar. Ao tempo de setup e de hold, juntos, damos o nome de **janela de estabilidade**. Esta janela ocorre, neste caso, algures entre a transição de estados low para high do clock, devidamente indicado na figura seguinte. Se não houver consistência entre tempos de setup e hold, o flip-flop muito provavelmente irá para um estado imprevisível: 0 ou 1. Em alguns casos, no entanto, a saída vai oscilar ou entra num estado meta-estável, como podemos constatar na penúltima área sombreada da figura. Se o flip-flop entrar em meta-estabilidade, ele retornará um estado estável por si próprio, após um atraso probabilístico. Também pode ser forçado a mudar para um estado estável, aplicando outro relógio com efeito *triggering-edge*, com uma entrada D que vai de encontro com os tempos de setup e hold, como representados no último sombreado da figura.

**janela de estabilidade**



**figura 50**  
diagrama temporal de um  
flip-flop D edge-triggered  
(em detalhe)

Os flip-flops podem estar ligados, assim, em cascata, pelo que constituem um bloco lógico importante em lógica sequencial: o **registo de deslocamento** (*shift register*). Uma grande vantagem desta lógica é o facto do funcionamento ser viabilizado por uma correta articulação dos parâmetros temporais, o que perfaz atrasos de propagação que excedem largamente os tempos de hold, o que garante que o “andar seguinte” guarde o valor (por muito que temporariamente) corrente antes que este seja substituído por um novo valor.

**registo de deslocamento**

Existem muitos outros flip-flops, sendo os mais interessantes para o estudo, os flip-flops JK e T (toogle).

Caracterização analítica

De forma a fazermos uma **caracterização analítica** de um sistema sequencial, um dos passos essenciais é a escrita e cálculo da **equação caraterística**.

Tomando o estado seguinte como função booleana do estado presente e das entradas de excitação de uma latch/flip-flop, conseguimos chegar à respetiva equação caraterística. Vejamos dois exemplos a partir das tabelas de transição:

|   |    |    |    |    |
|---|----|----|----|----|
|   |    |    | S  |    |
| Q | 00 | 01 | 11 | 10 |
| 0 | 0  | 0  | x  | 1  |
| 1 | 1  | 0  | x  | 1  |
|   |    |    | R  |    |

$Q^+ = S + Q\bar{R}$

|   |   |   |
|---|---|---|
| Q | 0 | 1 |
| 0 | 0 | 1 |
| 1 | 0 | 1 |

$Q^+ = D$

Para cada latch/flip-flop importa saber que estímulos devem ser aplicados para induzir uma determinada mudança de estado. A partir dos diagramas de estado, podemos construir as **tabelas de excitação**.

| Q→  | SR  | D |
|-----|-----|---|
| 0→0 | 0 X | 0 |
| 0→1 | 1 0 | 1 |
| 1→0 | 0 1 | 0 |
| 1→1 | X 0 | 1 |

Em suma, a seguinte tabela carateriza alguns dos comportamentos de alguns latches/flip-flops:

| tipo                             | amostragem das entradas                | validade das saídas                                      |
|----------------------------------|----------------------------------------|----------------------------------------------------------|
| latch                            | sempre                                 | atraso de propagação desde a alteração da entrada        |
| latch com clock (C)              | C = 1                                  | atraso de propagação desde a alteração da entrada        |
| flip-flop D edge-triggered (pos) | C = 0 para C = 1<br>T <sub>setup</sub> | atraso de propagação desde a transição low-high do clock |
| flip-flop D edge-triggered (neg) | C = 1 para C = 0<br>T <sub>setup</sub> | atraso de propagação desde a transição high-low do clock |
| flip-flop master slave           | C = 1<br>T <sub>setup</sub>            | atraso de propagação desde a transição high-low do clock |

**caraterização analítica**  
**equação caraterística**

**figura 51**  
equações caraterísticas a partir das tabelas de transição num K-map

**tabelas de excitação**

**tabela 14**  
tabela de excitação

**tabela 15**  
alguns comportamentos temporais de latches/flip-flops

## 9. Análise de sistemas sequenciais síncronos

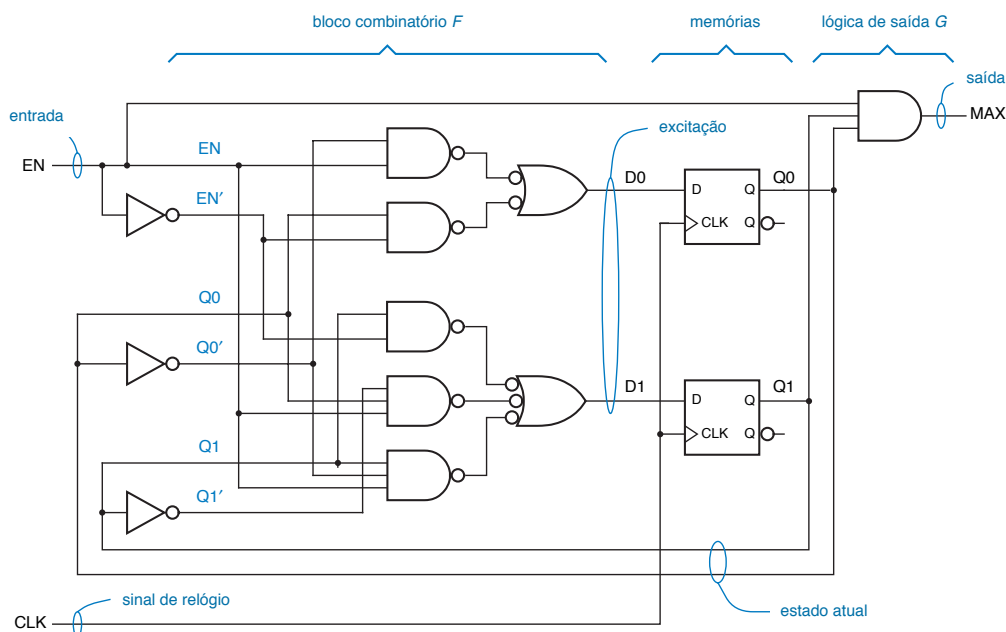
Quando nos apresentam um circuito sequencial, por forma de um esquema lógico dum sistema sequencial síncrono, como é que podemos caraterizar o seu funcionamento? Para fazer a sua caraterização nós devemos usar um vasto número de instrumentos como equações de excitação das estruturas elementares de memória (flip-flop), equações caraterísticas, tabelas de transição/saídas do sistema, diagramas de estado e diagramas temporais.

### Metodologia de análise

Vejam, desde já, uma boa ordem de análise de um sistema sequencial síncrono. Assim, de forma a analisar um circuito desta espécie, podemos seguir a seguinte metodologia:

- **Etapa 1** - etapa de preparação de valores e equações:
  - Para cada flip-flop determinar a função de excitação;
  - Usando a equação caraterística dos flip-flops, deduzir as equações de transição de estados (trivial com flip-flops do tipo D);
  - Determinar as equações de saída.
- **Etapa 2** - etapa de construção e pré-análise:
  - Construir uma tabela de transição de estados, indicando, para cada combinação estado/entrada, o estado seguinte;
  - Construir uma tabela de saídas, indicando, para cada combinação estado/entrada, os valores de saída (a qual pode ser combinada com a tabela de transição de estados).
- **Etapa 3** - etapa de análise completo de um sistema síncrono sequencial:
  - Desenhar o diagrama de estados.

Vejam, um exemplo de aplicação desta metodologia. Consideremos o seguinte esquema lógico de um determinado circuito síncrono sequencial.



**figura 52**  
sistema sequencial síncrono  
a analisar

# 49 INTRODUÇÃO AOS SISTEMAS DIGITAIS

Então, como tínhamos no nosso esquema metódico, a primeira etapa é de preparação de valores e equações, logo começemos por traduzir as equações de excitação, características, de transição e de saída.

equações de excitação:  $D_0 = Q_0 \cdot EN' + Q_0' \cdot EN$

$$D_1 = Q_1 \cdot EN' + Q_1' \cdot Q_0 \cdot EN + Q_1 \cdot Q_0' \cdot EN$$

equações características:  $Q_0^* = D_0$

$$Q_1^* = D_1$$

equações de transição:  $Q_0^* = Q_0 \cdot EN' + Q_0' \cdot EN$

$$Q_1^* = Q_1 \cdot EN' + Q_1' \cdot Q_0 \cdot EN + Q_1 \cdot Q_0' \cdot EN$$

equação de saída:  $MAX = Q_1 Q_0 EN$

**equação 53**

equações de excitação

**equação 54**

equações características

**equação 55**

equações de transição

**equação 56**

equação de saída

Se repararmos bem, as equações de excitação correspondem às do bloco combinatório  $F$  e as equações características aos flip-flops usados (memórias).

A segunda etapa serve para gerar uma pré-análise através de tabelas, entre as quais as de transições, estados e de estados/saídas.

| Q1 Q0   | EN  |     |
|---------|-----|-----|
|         | 0   | 1   |
| 0 0     | 0 0 | 0 1 |
| 0 1     | 0 1 | 1 0 |
| 1 0     | 1 0 | 1 1 |
| 1 1     | 1 1 | 0 0 |
| Q1* Q0* |     |     |

tabela de transições

| S  | EN |   |
|----|----|---|
|    | 0  | 1 |
| A  | A  | B |
| B  | B  | C |
| C  | C  | D |
| D  | D  | A |
| S* |    |   |

tabela de estados

| S       | EN   |      |
|---------|------|------|
|         | 0    | 1    |
| A       | A, 0 | B, 0 |
| B       | B, 0 | C, 0 |
| C       | C, 0 | D, 0 |
| D       | D, 0 | A, 1 |
| S*, MAX |      |      |

tabela de estados/saídas

**tabela 15**

tabela de transições

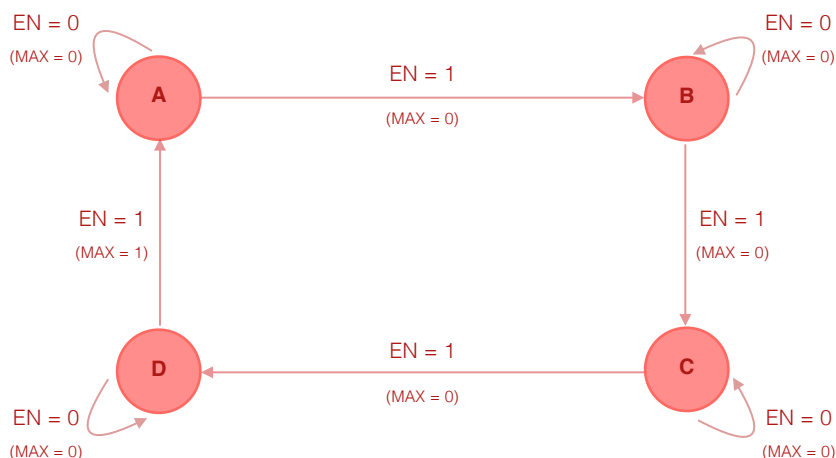
**tabela 16**

tabela de estados

**tabela 17**

tabela de estados/saídas

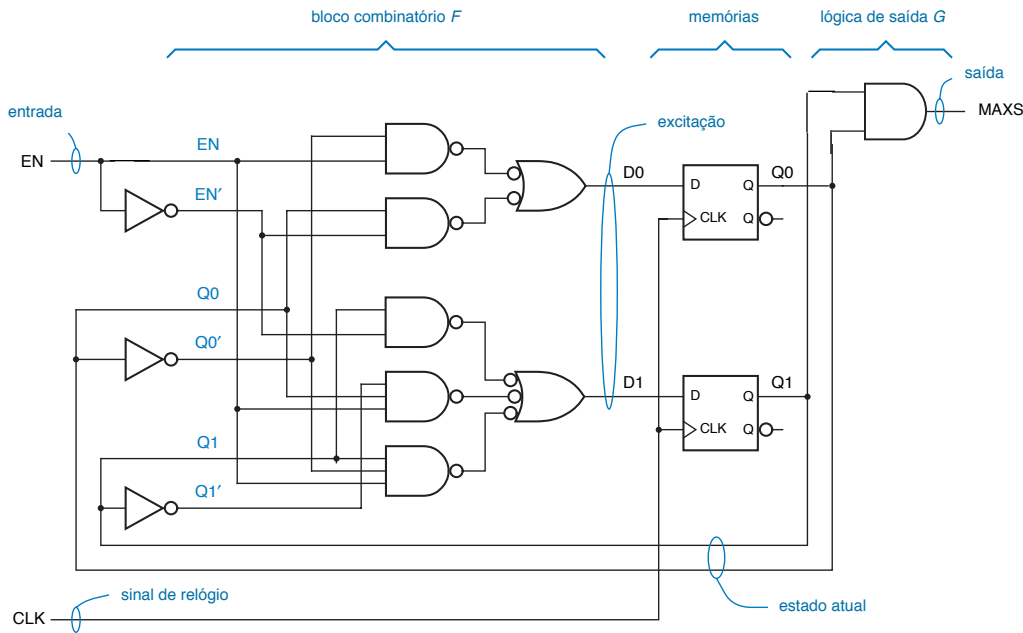
A terceira e última etapa completa a análise, através do desenho de um diagrama de estados.



**figura 53**

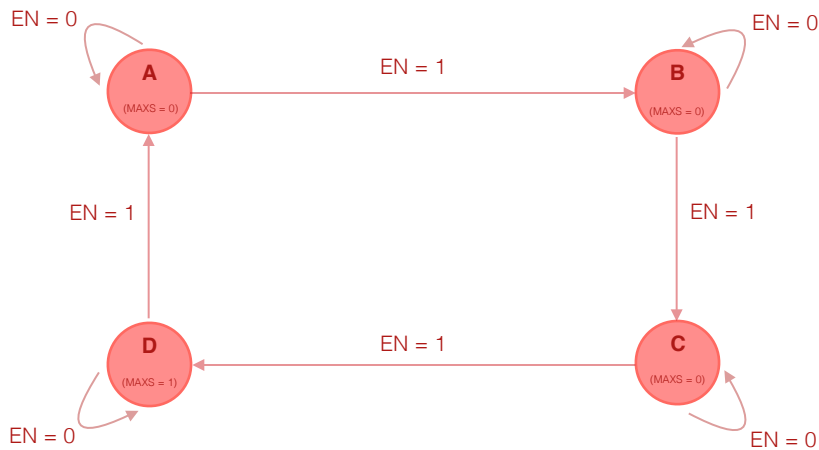
diagrama de estados

Façamos agora uma pequena variação do exemplo anterior. Para tal, consideremos o mesmo circuito com uma pequena alteração: MAX agora passa a MAXS (por questões de distinção dos casos); MAXS agora só conta com  $Q_0$  e  $Q_1$ , e não com EN. Vejamos o esquema.



**figura 54**  
re-análise do sistema anterior (modificado)

Vejamos agora o diagrama e tabela de estados/saídas.



**figura 55**  
diagrama de estados

|    | EN |   |      |
|----|----|---|------|
| S  | 0  | 1 | MAXS |
| A  | A  | B | 0    |
| B  | B  | C | 0    |
| C  | C  | D | 0    |
| D  | D  | A | 1    |
| S* |    |   |      |

**tabela 18**  
tabela de estados/saídas

Mas que diferenças estruturais existem entre este último exemplo e o anterior? No primeiro caso a saída como depende diretamente da entrada pode alterar-se de forma assíncrona. Já no segundo caso como a dependência é apenas relativa ao estado a saída varia de forma síncrona com o clock.

## Máquinas de Mealy e de Moore

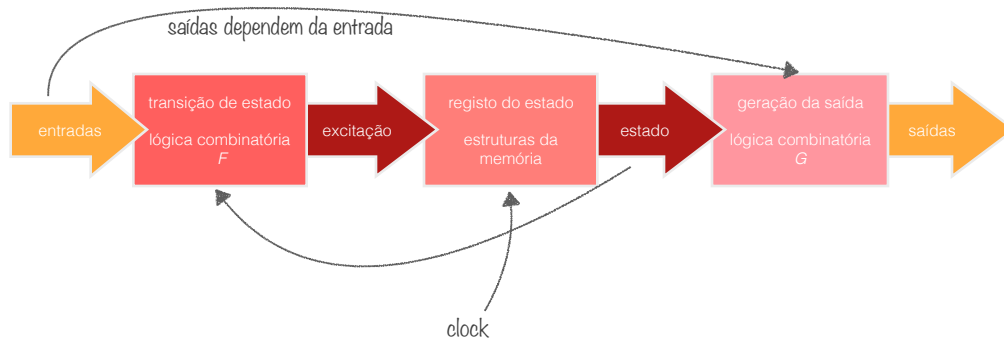
Esta pequena (grande) diferença no comportamento das saídas desta última constatação, permite-nos ilustrar, na prática, dois modelos de Máquinas de Estado Finitos (MEF): o **modelo de Mealy** e o **modelo de Moore**. O modelo ou máquina de Mealy tem saídas que podem ter comportamento assíncrono com o clock porque têm dependência combinatória com as entradas.

$$y_i = f[x_{n-1}, \dots, x_0, Q_{m-1}, \dots, Q_0]$$

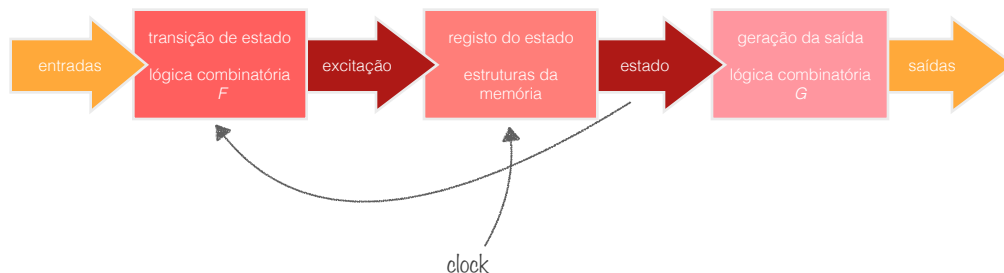
Por outro lado, a máquina de Moore tem saídas que têm um comportamento síncrono com relógio porque dependem exclusivamente das variáveis de estado.

$$y_i = f[Q_{m-1}, \dots, Q_0]$$

Vejamos, em esquema, o conceito da máquina de Mealy.



Vejamos também, em esquema, o conceito da máquina de Moore.



Se atentarmos nos diagramas e tabelas de estados/saídas, podemos reparar que existem várias diferenças entre ambos. Num modelo de Moore, o diagrama de estados contém, em cada estado, não só a designação do estado como também as saídas. No caso do modelo de Mealy apenas contem a identificação do estado. As próprias tabelas são diferentes.

**modelo de Mealy, modelo de Moore**

**equação 57**  
equação geral de uma máquina de Mealy

**equação 58**  
equação geral de uma máquina de Moore

**figura 56**  
esquema de funcionamento de uma máquina de Mealy

**figura 57**  
esquema de funcionamento de uma máquina de Moore

A máquina de Mealy tipicamente apresenta menos estados do que a máquina de Moore para a mesma sequência de saída.

## Limitações temporais

Dadas especificações temporais dos elementos de memória e dos tempos de propagação dos elementos combinatórios, coloca-se o problema de determinar qual a frequência máxima de funcionamento de um sistema sequencial síncrono.

Relembremos que os parâmetros temporais dos flip-flops são o tempo de setup, tempo de hold e os tempos de transição low para high e high para low. Já no caso na lógica combinatória associada à definição do estado seguinte (feedback) existem atrasos de propagação de portas elementares e ao nível do bloco.

Assim, temos que:

$$t_{CK} > t_{setup} + \max(t_{pHL}, t_{pLH}) + t_p(J = f(\dots), K = g(\dots), D = w(\dots))$$

estado seguinte

**equação 59**  
atrasos gerais de um equipamento

Vejam os seguintes exemplos. Consideremos o exemplo cuja saída final é MAX. Consideremos também que  $t_{setup} = 10ns$ ,  $t_{hold} = 5ns$ ,  $t_{pHL} = t_{pLH} = 20ns$ . O tempo de atraso das portas elementares é de 10ns.

$$T_{clock} > T_{setup} + T_{pHL} + 3T_p = 10 + 20 + 30 = 60ns$$

$$f_{clock} \leq \frac{1}{T_{clock}} = 16.6MHz$$

**equação 60**  
atrasos gerais de um equipamento (exemplo de aplicação)

## 10. Blocos sequenciais

Até agora vimos alguns circuitos sequenciais e até avançamos para a sua análise. Mas como último conteúdo desta disciplina, é importante referirmos dois últimos blocos sequenciais, entre os quais os contadores e os registros.

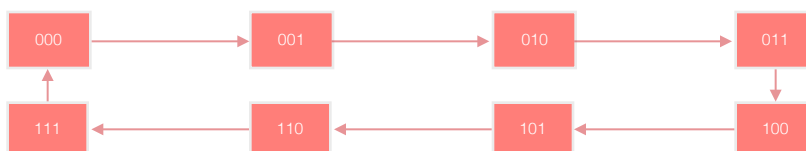
### Contadores

Os **contadores** são máquinas de estado finitos simples com um comportamento temporal periódico de módulo  $N$  em que  $N$  é o número de estados. Estes possuem, em análise, um diagrama de estados simples, com transições ao longo da sucessão ordenada de estados.

Tomemos o seguinte exemplo de contador: contador crescente de módulo 8 (3 bits). Um contador não necessita que se tome decisões sobre o estado seguinte, dado que a saída é o seu próprio estado presente. A codificação dos estados é inerente à própria sequência de contagem.

A metodologia de síntese destes equipamentos é semelhante à geral, definida anteriormente: fazer diagrama de estados, tabela de transições, escolha do flip-flop, tabela de excitações e equações de excitação.

Tomemos novamente, como exemplo de bloco sequencial em estudo, o contador crescente de módulo 8 (3 bits):



**contadores**

**figura 58**  
contador crescente de módulo 8 (3 bits)

# 53 INTRODUÇÃO AOS SISTEMAS DIGITAIS

A tabela de transições é a seguinte:

| Q2 | Q1 | Q0 | Q2+ | Q1+ | Q0+ |
|----|----|----|-----|-----|-----|
| 0  | 0  | 0  | 0   | 0   | 1   |
| 0  | 0  | 1  | 0   | 1   | 0   |
| 0  | 1  | 0  | 0   | 1   | 1   |
| 0  | 1  | 1  | 1   | 0   | 0   |
| 1  | 0  | 0  | 1   | 0   | 1   |
| 1  | 0  | 1  | 1   | 1   | 0   |
| 1  | 1  | 0  | 1   | 1   | 1   |
| 1  | 1  | 1  | 0   | 0   | 0   |

**tabela 19**  
tabela de transições

Vejamos agora um exemplo com flip-flops do tipo D. A tabela de excitações seria algo deste género.

| Q2 | Q1 | Q0 | Q2+ | Q1+ | Q0+ | D2 | D1 | D0 |
|----|----|----|-----|-----|-----|----|----|----|
| 0  | 0  | 0  | 0   | 0   | 1   | 0  | 0  | 1  |
| 0  | 0  | 1  | 0   | 1   | 0   | 0  | 1  | 0  |
| 0  | 1  | 0  | 0   | 1   | 1   | 0  | 1  | 1  |
| 0  | 1  | 1  | 1   | 0   | 0   | 1  | 0  | 0  |
| 1  | 0  | 0  | 1   | 0   | 1   | 1  | 0  | 1  |
| 1  | 0  | 1  | 1   | 1   | 0   | 1  | 1  | 0  |
| 1  | 1  | 0  | 1   | 1   | 1   | 1  | 1  | 1  |
| 1  | 1  | 1  | 0   | 0   | 0   | 0  | 0  | 0  |

**tabela 20**  
tabela de excitações

As equações de excitação são as seguintes:

| Q1       |    |    |    |    |   |
|----------|----|----|----|----|---|
| Q2 Q1 Q0 | 00 | 01 | 11 | 10 |   |
| Q0       | 0  | 1  | 0  | 0  | 1 |
| Q0       | 1  | 1  | 0  | 0  | 1 |

| Q1       |    |    |    |    |   |
|----------|----|----|----|----|---|
| Q2 Q1 Q0 | 00 | 01 | 11 | 10 |   |
| Q0       | 0  | 0  | 1  | 0  | 1 |
| Q0       | 1  | 0  | 1  | 0  | 1 |

| Q1       |    |    |    |    |   |
|----------|----|----|----|----|---|
| Q2 Q1 Q0 | 00 | 01 | 11 | 10 |   |
| Q0       | 0  | 0  | 0  | 1  | 0 |
| Q0       | 1  | 1  | 1  | 0  | 1 |

**figura 59**  
equações de excitação via  
K-map

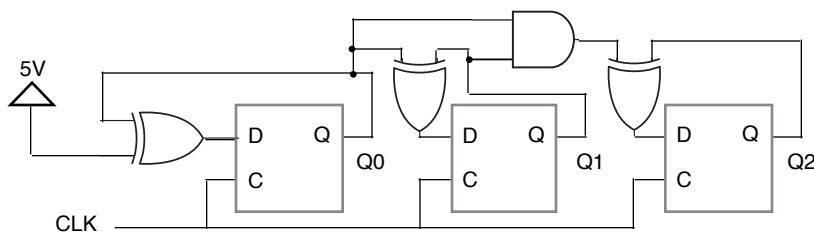
$$D0 = \bar{Q}0 = Q0 \oplus 1$$

$$D1 = Q1 \oplus Q0$$

$$D2 = Q2 \oplus Q1.Q0$$

Com  $N$  bits teríamos...  $Dn-1 = Qn-1 \oplus Qn-2...Q1.Q0$

Vejamos a implementação:



**equação 61**  
equação de excitação geral

**figura 60**  
diagrama lógico do contador

## Sequências de contagem não standard

Os contadores também podem efetuar contagens não completas ou irregulares. Vejamos um contador módulo 5.

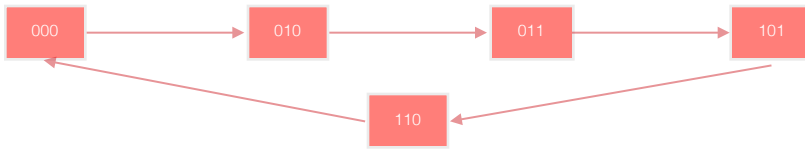


figura 61  
contador módulo 5

Se passarmos isto para uma tabela de excitações, obtemos o seguinte.

| Q2 | Q1 | Q0 | Q2+ | Q1+ | Q0+ |
|----|----|----|-----|-----|-----|
| 0  | 0  | 0  | 0   | 1   | 0   |
| 0  | 0  | 1  | x   | x   | x   |
| 0  | 1  | 0  | 0   | 1   | 1   |
| 0  | 1  | 1  | 1   | 0   | 1   |
| 1  | 0  | 0  | x   | x   | x   |
| 1  | 0  | 1  | 1   | 1   | 0   |
| 1  | 1  | 0  | 0   | 0   | 0   |
| 1  | 1  | 1  | x   | x   | x   |

tabela 21  
tabela de excitações

As equações de excitação são as seguintes:

$$D2 = Q0 \quad D1 = \bar{Q}1 + Q0\bar{Q}2 \quad D0 = \bar{Q}2Q1$$

equação 62  
equações de excitação

## Self-starting

No arranque de um contador (*power-up*) ele pode assumir qualquer estado, mesmo algum que não está incluído no diagrama de estados desejável. Mas o que é que acontece se o sistema cair num estado não incluído no diagrama inicial? É desejável que qualquer que seja o estado inicial ao fim de um qualquer número finito de contagens o contador entre no diagrama de estados previsto. Um sistema com esta capacidade diz-se um sistema com **self-starting**. Vejamos este exemplo:

self-starting

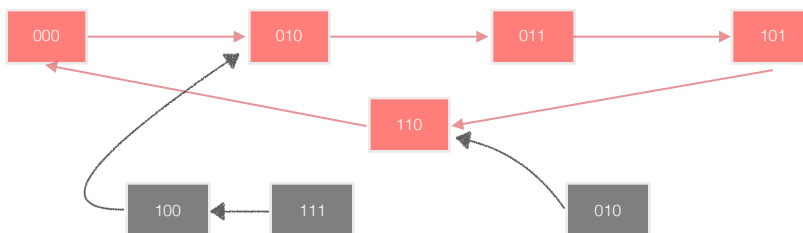


figura 62  
exemplo de sistema com  
self-starting

Neste exemplo prova-se, *a posteriori*, que o sistema goza de self-starting.

## Contadores MSI

Os **contadores MSI** são blocos sequenciais de contagem com complexidade média. As principais vantagens da sua utilização são a sua versatilidade, a capacidade de programação paralela, ter um clear síncrono ou assíncrono, conter modos de hold e obter “flags” de fim de contagem. Os dois componentes MSI em estudo são os 74x161 e o 74x163. Estes contadores têm um processo de contagem muito próprio.

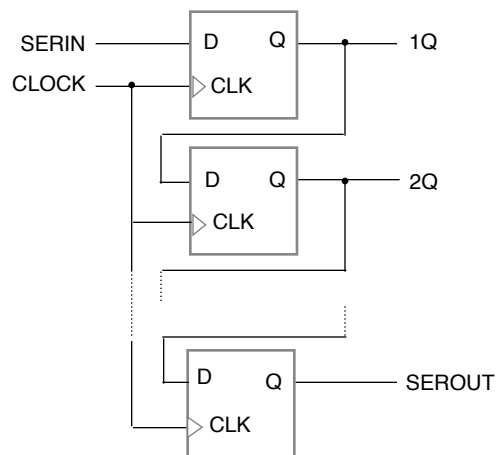
O contador componente 74x163 tem um modo de contagem “normal”, com um aviso de fim de contagem.

**contadores MSI**

## Registos de deslocamento

Um **registo de deslocamento** é um registo de  $n$  bits com preparação para deslocar dados guardados em uma posição a cada pulso de relógio. A figura abaixo mostra a estrutura de um registo do tipo serial-in-serial-out. A entrada serial, SERIN, especifica um novo bit a ser deslocado para uma saída a cada pulso do relógio. Este bit aparece então, na saída do nosso sistema, SEROUT, depois de  $n$  pulsos de relógio, e perde-se um pulso depois. Assim, um registo serial-in-serial-out de  $n$  bits pode ser usado para atrasar um sinal  $n$  pulsos de relógio.

**registo de deslocamento**

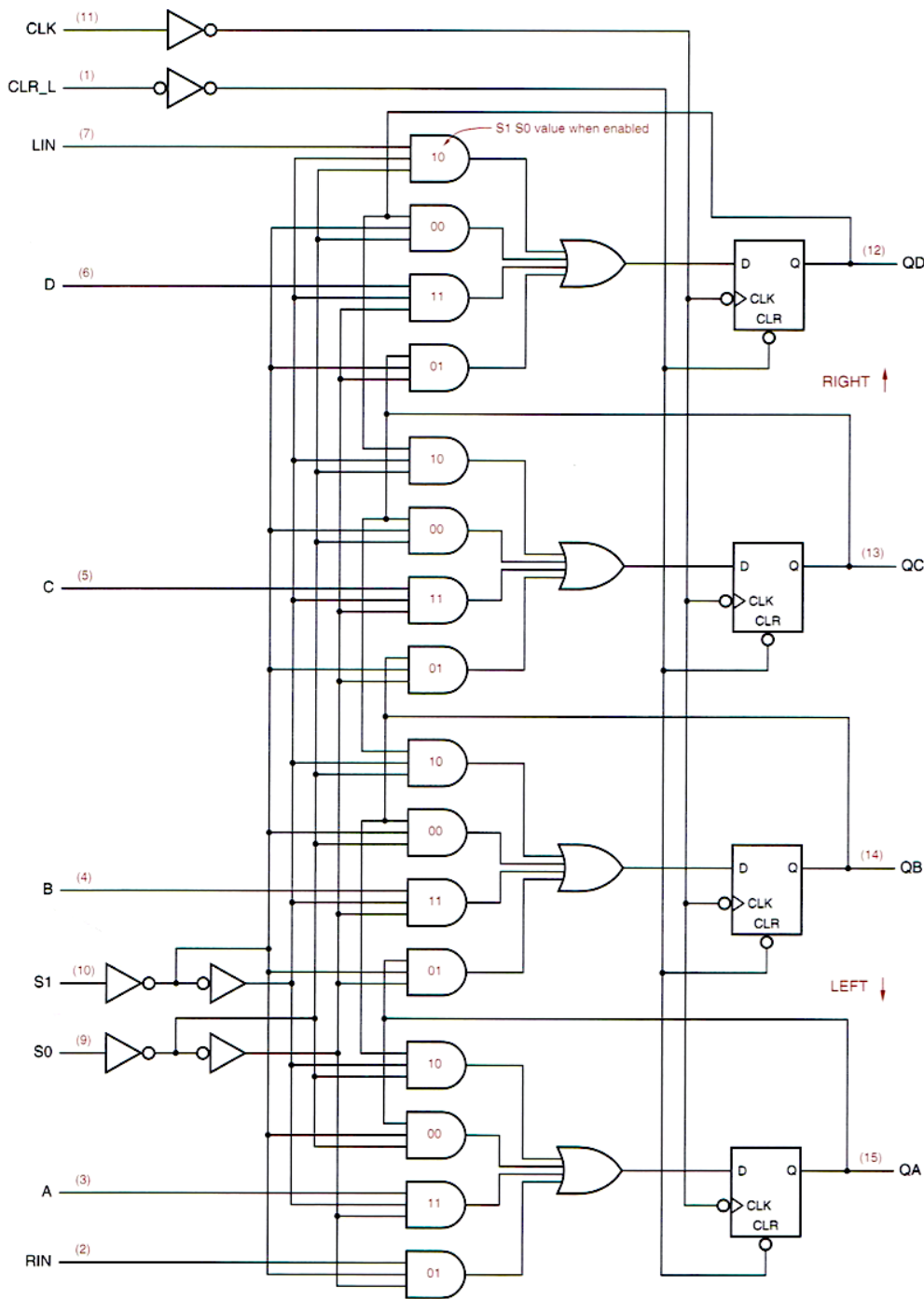


**figura 63**

**registo serial-in-serial-out**

As famílias tradicionais de lógicas MSI fornecem entre muitos outros, registos de deslocamento de 4 e 8 bits. Estas partes MSI são usadas muito raramente, nos dias que correm, porque qualquer outro tipo de registo de deslocamento pode ser fornecido através de PLD ou FPGA (assuntos a abordar na disciplina de Laboratório de Sistemas Digitais (als2)). No entanto, ser-nos-á útil olhar para uma parte MSI, de forma a obter uma ideia sobre a complexidade de um registo de deslocamento e do seu detalhe interno.

O componente 74x194 é um registo MSI de 4 bits, bidirecional, do tipo parallel-in-parallel-out. O seu diagrama lógico está expresso na figura seguinte (da página seguinte). O componente da figura anterior era unidirecional porque fazia deslocamento num e num só sentido. O 74x194 é um componente bidirecional porque o seu conteúdo pode ser deslocado em qualquer uma de duas direções, dependendo da entrada. As duas direções têm os nomes de “esquerda” (*left*) e “direita” (*right*), mesmo que o esquema possa não estar desenhado dessa forma. Neste componente em específico, esquerda significa na direção de QD para QA, e direita significa na direção de QA para QD.



**figura 64**  
registro  
parallel-in-parallel-out

A tabela na página seguinte é uma tabela de verdade para o componente 74x194. Esta tabela está altamente comprimida, dado que não contém colunas para a maior parte das entradas (A-D, RIN, LIN) ou para o estado atual (QA-QD). Mesmo assim, exprimindo cada estado seguinte como uma função implícita dessas variáveis, já conseguimos definir a operação do componente para todas as  $2^{12}$  combinações possíveis de estados atuais e entradas, o que quase de certeza que ia além das 4096 linhas numa tabela!

| inputs      |    |    | estados seguintes |     |     |     |
|-------------|----|----|-------------------|-----|-----|-----|
| função      | S1 | S2 | QA*               | QB* | QC* | QD* |
| hold        | 0  | 0  | QA                | QB  | QC  | QD  |
| shift right | 0  | 1  | RIN               | QA  | QB  | QC  |
| shift left  | 1  | 0  | QB                | QC  | QD  | LIN |
| load        | 1  | 1  | A                 | B   | C   | D   |

**tabela 22**

**tabela de verdade do componente 74x194**

Note-se que a entrada LIN (*left-in*) está concetualmente localizada na parte direita do chip, mas é a entrada serial para deslocamentos para a esquerda. Isto também se aplica à entrada RIN.

O componente 74x194 é também chamado, muitas vezes, de **registo de deslocamento universal** porque pode ser usado para funcionar como muitos outros registo a ele inferiores.

Outros registos de deslocamento são os de contadores, como os contadores com feedback em **anel** e o de **Johnson** (também conhecido como **twisted ring**).

E assim termina o programa de Introdução aos Sistemas Digitais. Para continuar a estudar, no seguimento deste documento vem uma de duas disciplinas. Dependendo do percurso curricular, é possível escolher entre os apontamentos de Arquitetura de Sistemas Operativos ou Laboratório de Sistemas Digitais, ambos do primeiro ano, segundo semestre (als2).

**registo de deslocamento universal**

**anel, Johnson, twisted ring**

## Ambiente Digital

|                                  |   |
|----------------------------------|---|
| Analógico versus Digital .....   | 2 |
| Álgebra de Boole .....           | 3 |
| Portas lógicas e flip-flops..... | 4 |

## Sistemas numéricos e códigos

|                                                |    |
|------------------------------------------------|----|
| Elementos de um sistema de numeração .....     | 6  |
| Mudanças de base .....                         | 8  |
| Adição e Subtração binária .....               | 9  |
| Multiplicação binária.....                     | 10 |
| Representação de quantidades negativas.....    | 10 |
| Sinal e módulo .....                           | 10 |
| Complemento para a base r .....                | 11 |
| Complemento para a base 2 .....                | 11 |
| Overflow .....                                 | 12 |
| Adição algébrica em complemento para um .....  | 13 |
| Adição algébrica em complemento para dois..... | 13 |

## Códigos

|                                            |    |
|--------------------------------------------|----|
| BCD (Binary Coded Decimal).....            | 13 |
| Código AIKEN e Excess-3 (XS3) .....        | 14 |
| Código de Gray .....                       | 14 |
| Deteção de erros em sistemas digitais..... | 15 |
| Códigos CRC .....                          | 16 |

## Álgebra de Boole

|                                        |    |
|----------------------------------------|----|
| Postulados .....                       | 16 |
| Valores, Operadores e Expressões ..... | 17 |
| Dualidade.....                         | 17 |
| Teoremas.....                          | 18 |
| Operadores compostos .....             | 18 |
| Funções booleanas.....                 | 19 |
| Formas canónicas.....                  | 20 |

## Minimização de funções booleanas

|                                                          |    |
|----------------------------------------------------------|----|
| Expressões irreduzíveis .....                            | 23 |
| Adjacência e simplificação .....                         | 23 |
| Cubos Booleanos.....                                     | 23 |
| Métodos dos mapas de Karnaugh (K-map).....               | 25 |
| Minimização com don't care's.....                        | 27 |
| Estratégia de cobertura .....                            | 27 |
| Algoritmo de cobertura para soma de produtos mínima..... | 28 |

## Blocos Combinatórios

|                                     |    |
|-------------------------------------|----|
| Lógica de Polaridade.....           | 29 |
| Descodificadores (decoders) .....   | 29 |
| Codificadores (encoder) .....       | 30 |
| Multiplexers e demultiplexers ..... | 31 |
| Buffers 3-state .....               | 32 |

## Circuitos Aritméticos

|                                         |    |
|-----------------------------------------|----|
| Circuitos somadores .....               | 33 |
| Somadores carry-lookahead.....          | 35 |
| Somadores BCD .....                     | 36 |
| Unidade aritmética e lógica (ALU) ..... | 37 |
| Multiplicação de números binários ..... | 38 |

### Sistemas Sequenciais

|                                                    |    |
|----------------------------------------------------|----|
| Lógica sequencial versus lógica combinatória ..... | 39 |
| Estruturas elementares de memória .....            | 39 |
| Latch SR.....                                      | 40 |
| Latch SR com portas NAND .....                     | 42 |
| Latch SR com enable.....                           | 43 |
| Latch D.....                                       | 43 |
| Flip-flops .....                                   | 45 |
| Flip-flop D edge-triggered .....                   | 45 |
| Caracterização analítica .....                     | 47 |

### Análise de sistemas sequenciais síncronos

|                                    |    |
|------------------------------------|----|
| Metodologia de análise.....        | 48 |
| Máquinas de Mealy e de Moore ..... | 51 |
| Limitações temporais .....         | 52 |

### Blocos sequenciais

|                                          |    |
|------------------------------------------|----|
| Contadores.....                          | 52 |
| Sequências de contagem não standard..... | 54 |
| Self-starting .....                      | 54 |
| Contadores MSI .....                     | 55 |
| Registos de deslocamento .....           | 55 |

